

MODULE 4

This Chapter covers

- Basic building blocks of an IoT Device
- Exemplary Device: Raspberry Pi
- Raspberry Pi interfaces
- Programming Raspberry Pi with Python
- Other IoT devices

7.1 What is an IoT Device

As described earlier, a "Thing" in Internet of Things (IoT) can be any object that has a unique identifier and which can send/receive data (including user data) over a network (e.g., smart phone, smart TV, computer, refrigerator, car, etc.). IoT devices are connected to the Internet and send information about themselves or about their surroundings (e.g. information sensed by the connected sensors) over a network (to other devices or servers/storage) or allow actuation upon the physical entities/environment around them remotely. Some examples of IoT devices are listed below:

- A home automation device that allows remotely monitoring the status of appliances and controlling the appliances.
- An industrial machine which sends information about its operation and health monitoring data to a server.
- A car which sends information about its location to a cloud-based service.
- A wireless-enabled wearable device that measures data about a person such as the number of steps walked and sends the data to a cloud-based service.

7.1.1 Basic building blocks of an IoT Device

An IoT device can consist of a number of modules based on functional attributes, such as:

- Sensing: Sensors can be either on-board the IoT device or attached to the device. IoT device can collect various types of information from the on-board or attached sensors such as temperature, humidity, light intensity, etc. The sensed information can be communicated either to other devices or cloud-based servers/storage.
- Actuation: IoT devices can have various types of actuators attached that allow taking actions upon the physical entities in the vicinity of the device. For example, a relay switch connected to an IoT device can turn an appliance on/off based on the commands sent to the device.
- Communication: Communication modules are responsible for sending collected data to other devices or cloud-based servers/storage and receiving data from other devices and commands from remote applications.
- Analysis & Processing: Analysis and processing modules are responsible for making sense of the collected data.

The representative IoT device used for the examples in this book is the widely used single-board mini computer called Raspberry Pi (explained in later sections). The use of Raspberry Pi is intentional since these devices are widely accessible, inexpensive, and available from multiple vendors. Furthermore, extensive information is available on their programming and use both on the Internet and in other textbooks. The principles we teach in this book are just as applicable to other (including proprietary) IoT endpoints, in addition to Raspberry Pi. Before we look at the specifics of Raspberry Pi, let us first look at the building blocks of a generic single-board computer (SBC) based IoT device.

Figure 7.1 shows a generic block diagram of a single-board computer (SBC) based IoT device that includes CPU, GPU, RAM, storage and various types of interfaces and peripherals.

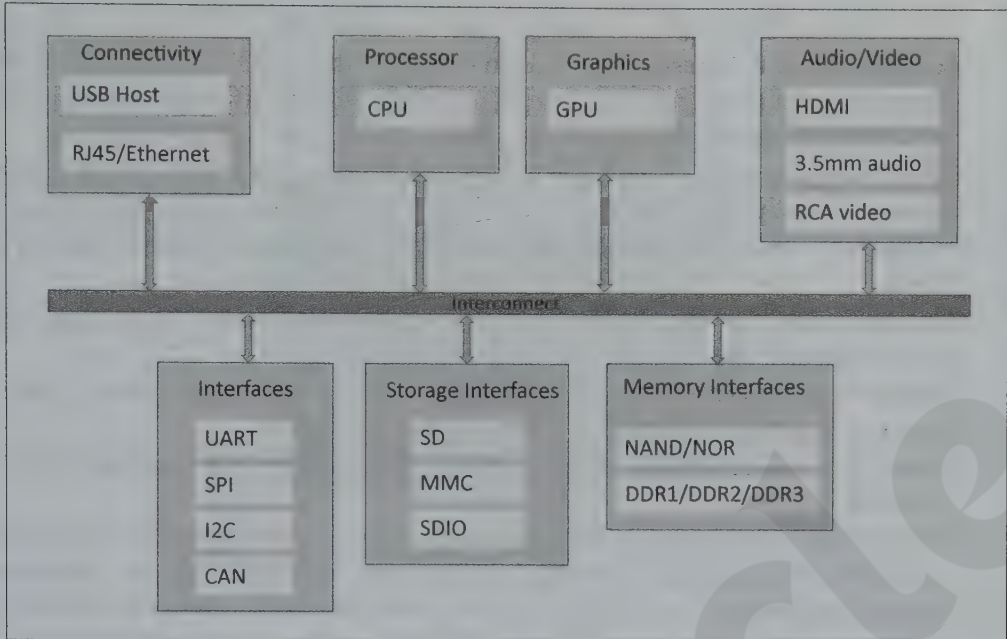


Figure 7.1: Block diagram of an IoT Device

7.2 Exemplary Device: Raspberry Pi

Raspberry Pi [104] is a low-cost mini-computer with the physical size of a credit card. Raspberry Pi runs various flavors of Linux and can perform almost all tasks that a normal desktop computer can do. In addition to this, Raspberry Pi also allows interfacing sensors and actuators through the general purpose I/O pins. Since Raspberry Pi runs Linux operating system, it supports Python "out of the box".

7.3 About the Board

Figure 7.2 shows the Raspberry Pi board with the various components/peripherals labeled.

- **Processor & RAM** : Raspberry Pi is based on an ARM processor. The latest version of Raspberry Pi (Model B, Revision 2) comes with 700 MHz Low Power ARM1176JZ-F processor and 512 MB SDRAM.
- **USB Ports** : Raspberry Pi comes with two USB 2.0 ports. The USB ports on Raspberry Pi can provide a current upto 100mA. For connecting devices that draw current more than 100mA, an external USB powered hub is required.
- **Ethernet Ports** : Raspberry Pi comes with a standard RJ45 Ethernet port. You can connect an Ethernet cable or a USB Wifi adapter to provide Internet connectivity.
- **HDMI Output** : The HDMI port on Raspberry Pi provides both video and audio output. You can connect the Raspberry Pi to a monitor using an HDMI cable. For monitors that have a DVI port but no HDMI port, you can use an HDMI to DVI adapter/cable.
- **Composite Video Output** : Raspberry Pi comes with a composite video output with an RCA jack that supports both PAL and NTSC video output. The RCA jack can be

used to connect old televisions that have an RCA input only.

- **Audio Output** : Raspberry Pi has a 3.5mm audio output jack. This audio jack is used for providing audio output to old televisions along with the RCA jack for video. The audio quality from this jack is inferior to the HDMI output.
- **GPIO Pins** : Raspberry Pi comes with a number of general purpose input/output pins. Figure 7.3 shows the Raspberry Pi GPIO headers. There are four types of pins on Raspberry Pi - true GPIO pins, I2C interface pins, SPI interface pins and serial Rx and Tx pins.
- **Display Serial Interface (DSI)** : The DSI interface can be used to connect an LCD panel to Raspberry Pi.
- **Camera Serial Interface (CSI)** : The CSI interface can be used to connect a camera module to Raspberry Pi.
- **Status LEDs** : Raspberry Pi has five status LEDs. Table 7.1 lists Raspberry Pi status LEDs and their functions.
- **SD Card Slot** : Raspberry Pi does not have a built in operating system and storage. You can plug-in an SD card loaded with a Linux image to the SD card slot. Appendix-A provides instructions on setting up New Out-of-the-Box Software (NOOBS) on Raspberry Pi. You will require atleast an 8GB SD card for setting up NOOBS.
- **Power Input** : Raspberry Pi has a micro-USB connector for power input.

Status LED	Function
ACT	SD card access
PWR	3.3V Power is present
FDX	Full duplex LAN connected
LNK	Link/Network activity
100	100 Mbit LAN connected

Table 7.1: Raspberry Pi Status LEDs

7.4 Linux on Raspberry Pi

Raspberry Pi supports various flavors of Linux including:

- **Raspbian** Raspbian Linux is a Debian Wheezy port optimized for Raspberry Pi. This is the recommended Linux for Raspberry Pi. Appendix-1 provides instructions on setting up Raspbian on Raspberry Pi.
- **Arch** : Arch is an Arch Linux port for AMD devices.
- **Pidora** : Pidora Linux is a Fedora Linux optimized for Raspberry Pi.
- **RaspBMC** : RaspBMC is an XBMC media-center distribution for Raspberry Pi.
- **OpenELEC** : OpenELEC is a fast and user-friendly XBMC media-center distribution.
- **RISC OS** : RISC OS is a very fast and compact operating system.

Figure 7.4 shows the Raspbian Linux desktop on Raspberry Pi. Figure 7.5 shows the default file explorer on Raspbian. Figure 7.6 shows the default console on Raspbian. Figure 7.7 shows the default browser on Raspbian. To configure Raspberry Pi, the raspi-config tool is used which can be launched from command line as (\$raspi-config) as shown in

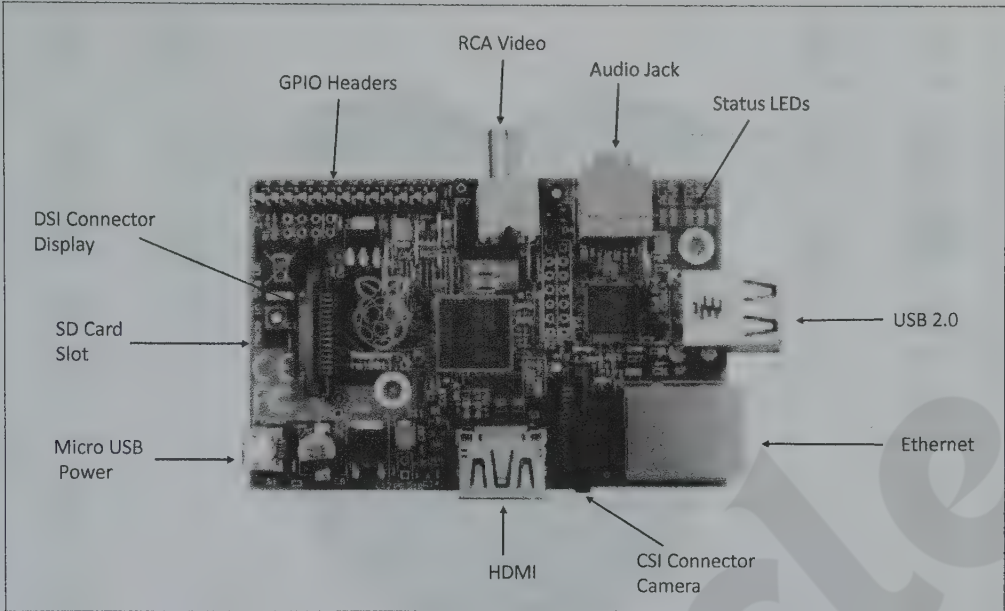


Figure 7.2: Raspberry Pi board

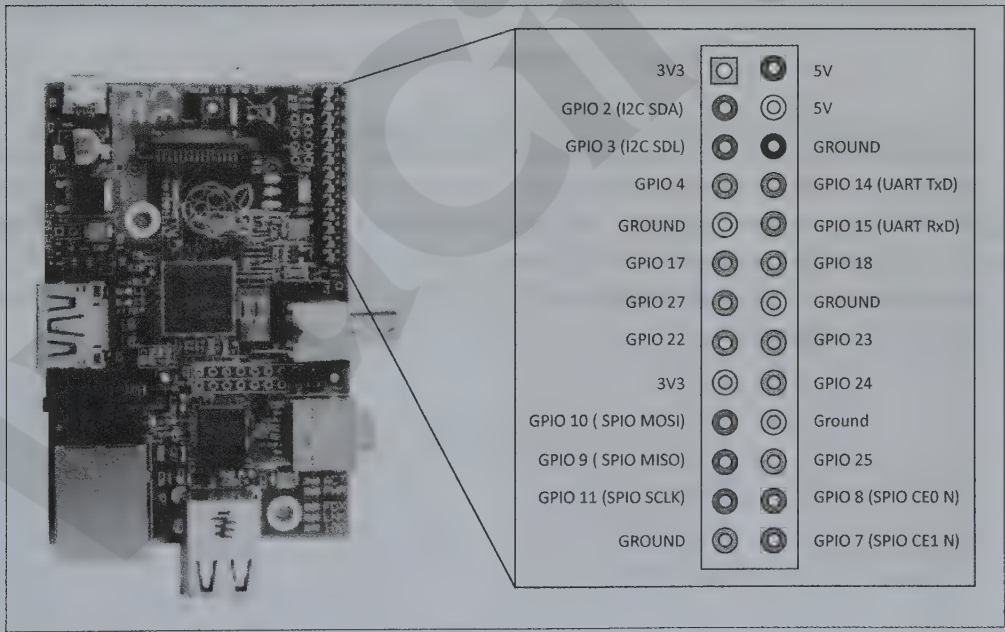


Figure 7.3: Raspberry Pi GPIO headers

Figure 7.8. Using the configuration tool you can expand root partition to fill SD card, set keyboard layout, change password, set locale and timezone, change memory split, enable or disable SSH server and change boot behavior. It is recommended to expand the root file-system so that you can use the entire space on the SD card.

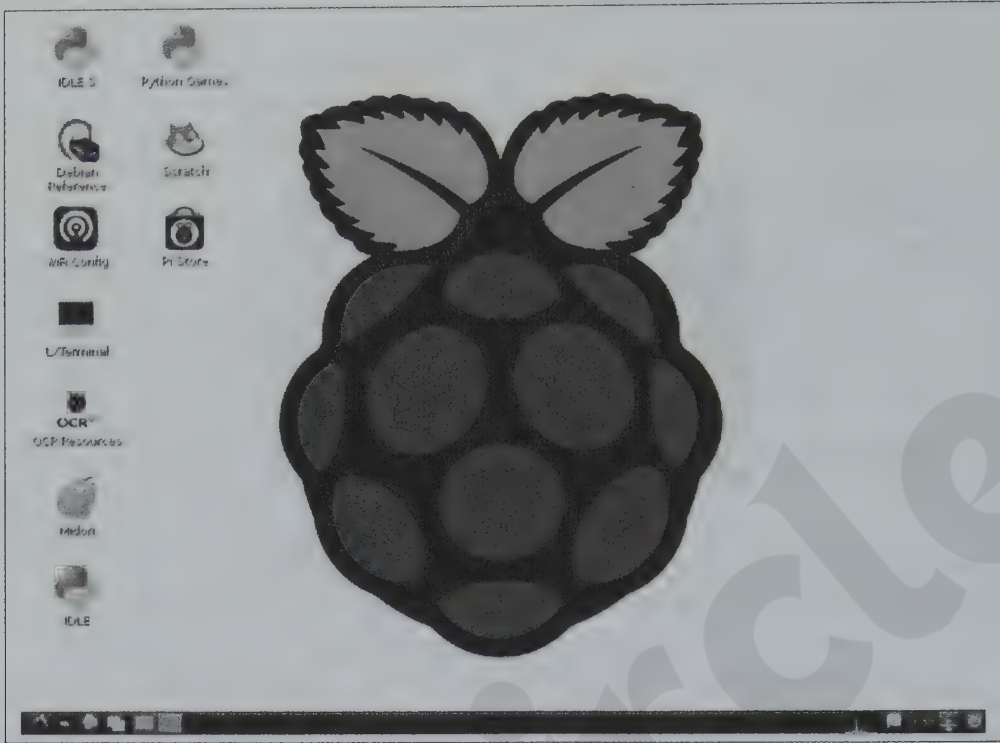


Figure 7.4: Raspbian Linux desktop

Though Raspberry Pi comes with an HDMI output, it is more convenient to access the device with a VNC connection or SSH. This does away with the need for a separate display for Raspberry Pi and you can use Raspberry Pi from your desktop or laptop computer. Appendix-A provides instructions on setting up VNC server on Raspberry Pi and the instructions to connect to Raspberry Pi with SSH. Table 7.2 lists the frequently used commands on Raspberry Pi.

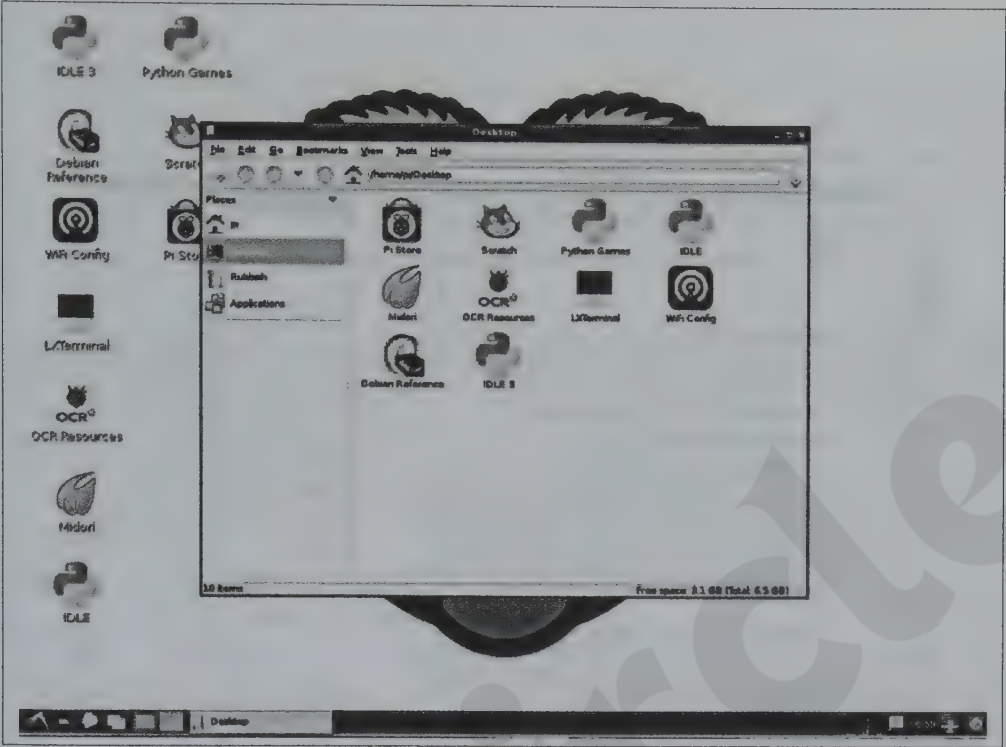


Figure 7.5: File explorer on Raspberry Pi



Figure 7.6: Console on Raspberry Pi

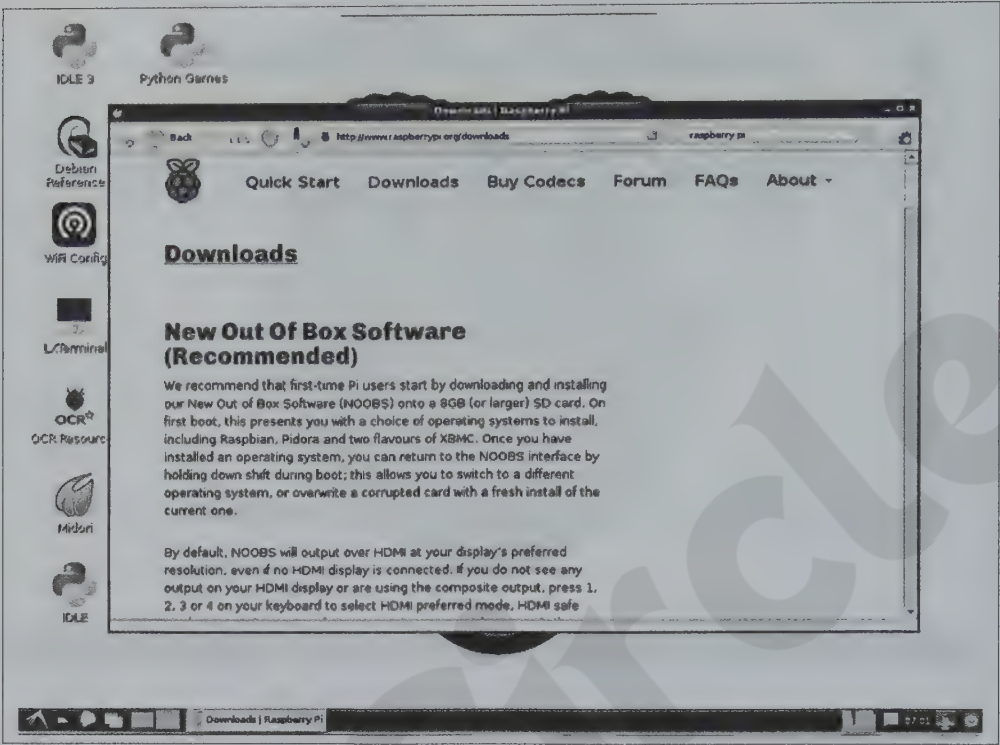


Figure 7.7: Browser on Raspberry Pi

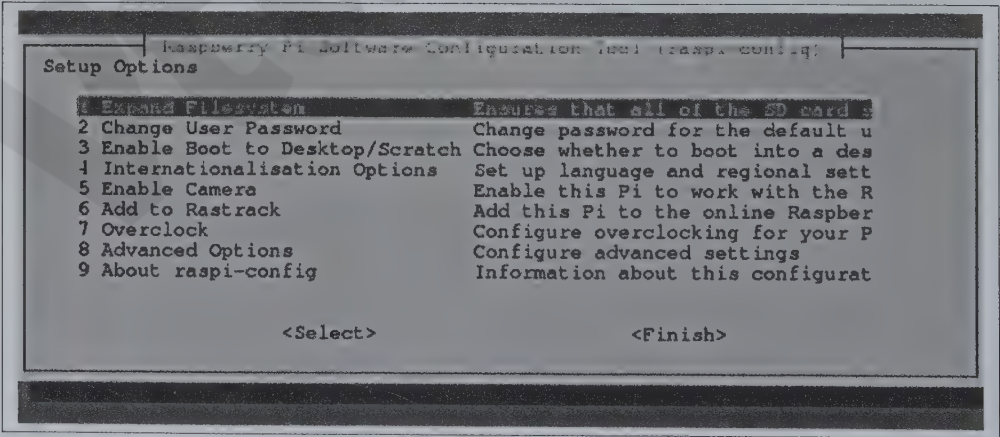


Figure 7.8: Raspberry Pi configuration tool

Command	Function	Example
cd	Change directory	cd /home/pi
cat	Show file contents	cat file.txt
ls	List files and folders	ls /home/pi
locate	Search for a file	locate file.txt
lsusb	List USB devices	lsusb
pwd	Print name of present working directory	pwd
mkdir	Make directory	mkdir /home/pi/new
mv	Move (rename) file	mv sourceFile.txt destinationFile.txt
rm	Remove file	rm file.txt
reboot	Reboot device	sudo reboot
shutdown	Shutdown device	sudo shutdown -h now
grep	Print lines matching a pattern	grep -r "pi" /home/
df	Report file system disk space usage	df -Th
ifconfig	Configure a network interface	ifconfig
netstat	Print network connections, routing tables, interface statistics	netstat -lntp
tar	Extract/create archive	tar -xzf foo.tar.gz
wget	Non-interactive network downloader	wget http://example.com/file.tar.gz

Table 7.2: Raspberry Pi frequently used commands

7.5 Raspberry Pi Interfaces

Raspberry Pi has serial, SPI and I2C interfaces for data transfer as shown in Figure 7.3.

7.5.1 Serial

The serial interface on Raspberry Pi has receive (Rx) and transmit (Tx) pins for communication with serial peripherals.

7.5.2 SPI

Serial Peripheral Interface (SPI) is a synchronous serial data protocol used for communicating with one or more peripheral devices. In an SPI connection, there is one master device and one or more peripheral devices. There are five pins on Raspberry Pi for SPI interface:

- **MISO (Master In Slave Out)** : Master line for sending data to the peripherals.
- **MOSI (Master Out Slave In)** : Slave line for sending data to the master.
- **SCK (Serial Clock)** : Clock generated by master to synchronize data transmission
- **CE0 (Chip Enable 0)** : To enable or disable devices.
- **CE0 (Chip Enable 1)** : To enable or disable devices.

7.5.3 I2C

The I2C interface pins on Raspberry Pi allow you to connect hardware modules. I2C interface allows synchronous data transfer with just two pins - SDA (data line) and SCL (clock line).

7.6 Programming Raspberry Pi with Python

In this section you will learn how to get started with developing Python programs on Raspberry Pi. Raspberry Pi runs Linux and supports Python out of the box. Therefore, you can run any Python program that runs on a normal computer. However, it is the general purpose input/output capability provided by the GPIO pins on Raspberry Pi that makes it useful device for Internet of Things. You can interface a wide variety of sensor and actuators with Raspberry Pi using the GPIO pins and the SPI, I2C and serial interfaces. Input from the sensors connected to Raspberry Pi can be processed and various actions can be taken, for instance, sending data to a server, sending an email, triggering a relay switch.

7.6.1 Controlling LED with Raspberry Pi

Let us start with a basic example of controlling an LED from Raspberry Pi. Figure 7.9 shows the schematic diagram of connecting an LED to Raspberry Pi. Box 7.1 shows how to turn the LED on/off from command line. In this example the LED is connected to GPIO pin 18. You can connect the LED to any other GPIO pin as well.

Box 7.2 shows a Python program for blinking an LED connected to Raspberry Pi every second. The program uses the RPi.GPIO module to control the GPIO on Raspberry Pi. In this program we set pin 18 direction to output and then write *True/False* alternatively after a delay of one second.

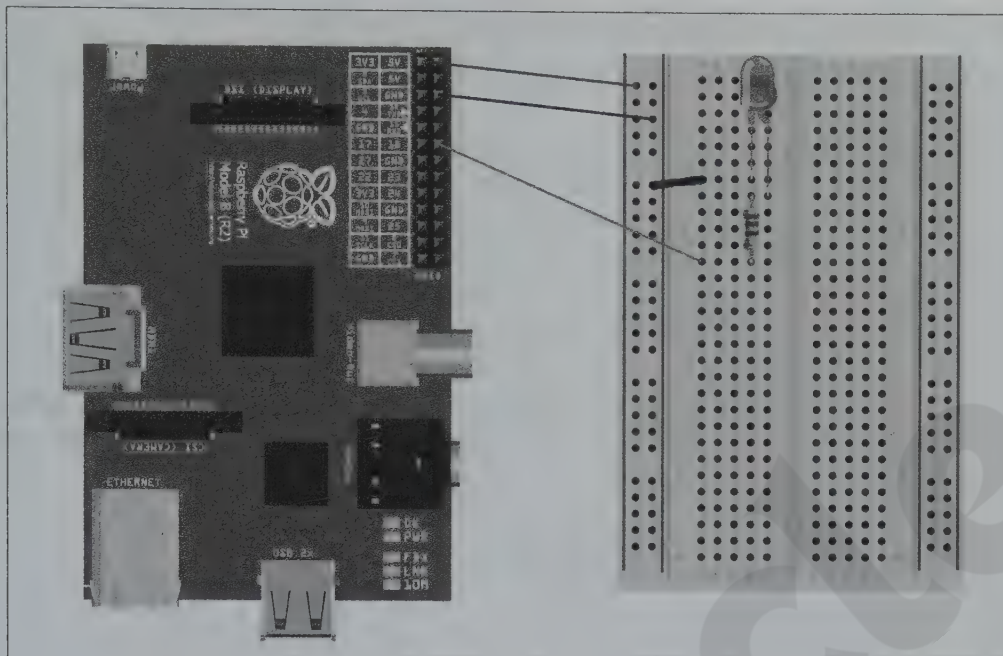


Figure 7.9: Controlling LED with Raspberry Pi

■ Box 7.1: Switching LED on/off from Raspberry Pi console

```
$echo 18 > /sys/class/gpio/export
$cd /sys/class/gpio/gpio18
```

```
#Set pin 18 direction to out
$echo out > direction
```

```
#Turn LED on
$echo 1 > value
#Turn LED off
$echo 0 > value
```

■ Box 7.2: Python program for blinking LED

```
import RPi.GPIO as GPIO
import time

GPIO.setmode(GPIO.BCM)
GPIO.setup(18, GPIO.OUT)

while True:
    GPIO.output(18, True)
    time.sleep(1)
    GPIO.output(18, False)
    time.sleep(1)
```

7.6.2 Interfacing an LED and Switch with Raspberry Pi

Now let us look at a more detailed example involving an LED and a switch that is used to control the LED.

Figure 7.10 shows the schematic diagram of connecting an LED and switch to Raspberry Pi. Box 7.3 shows a Python program for controlling an LED with a switch. In this example the LED is connected to GPIO pin 18 and switch is connected to pin 25. In the infinite while loop the value of pin 25 is checked and the state of LED is toggled if the switch is pressed. This example shows how to get input from GPIO pins and process the input and take some action. The action in this example is toggling the state of an LED. Let us look at another example, in which the action is an email alert. Box 7.4 shows a Python program for sending an email on switch press. Note that the structure of this program is similar to the program in Box 7.3. This program uses the Python SMTP library for sending an email when the switch connected to Raspberry Pi is pressed.

■ Box 7.3: Python program for controlling an LED with a switch

```
from time import sleep
import RPi.GPIO as GPIO

GPIO.setmode(GPIO.BCM)

#Switch Pin
GPIO.setup(25, GPIO.IN)

#LED Pin
GPIO.setup(18, GPIO.OUT)

state=False

def toggleLED(pin):
    state = not state
    GPIO.output(pin, state)

while True:
    try:
        if (GPIO.input(25) == True):
            toggleLED(pin)
            sleep(.01)
    except KeyboardInterrupt:
        exit()
```

■ Box 7.4: Python program for sending an email on switch press

```
import smtplib
from time import sleep
import RPi.GPIO as GPIO
from sys import exit
```

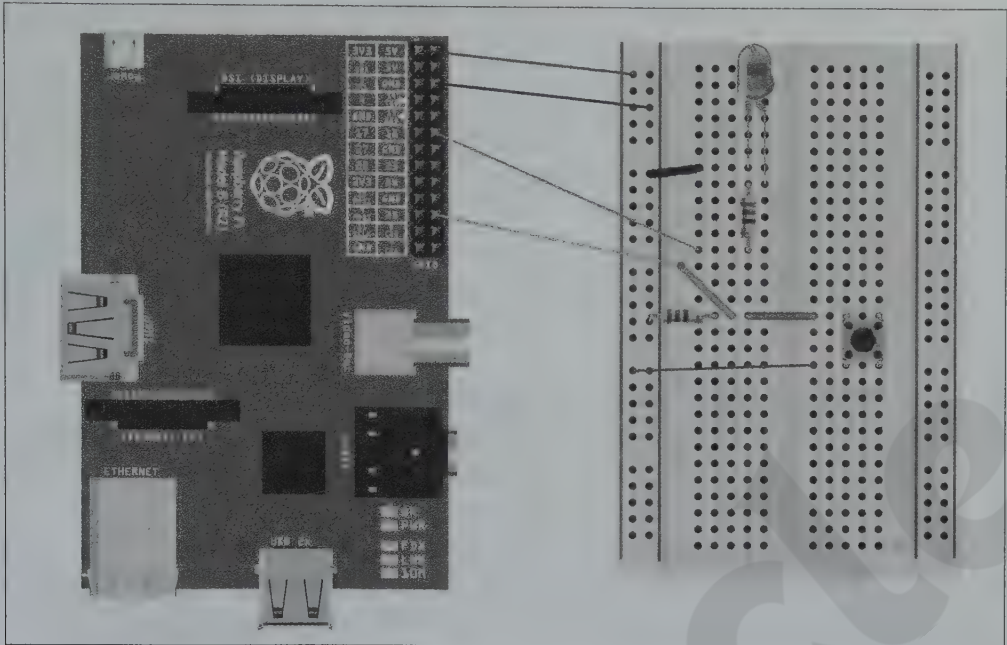


Figure 7.10: Interfacing LED and switch with Raspberry Pi

```

from_email = '<my-email>'
receipients_list = ['<receipient-email>']
cc_list = []
subject = 'Hello'
message = 'Switch pressed on Raspberry Pi'
username = '<Gmail-username>'
password = '<password>'
server = 'smtp.gmail.com:587'

GPIO.setmode(GPIO.BCM)
GPIO.setup(25, GPIO.IN)

def sendemail(from_addr, to_addr_list, cc_addr_list,
              subject, message,
              login, password,
              smtpserver):

    header = 'From: %s \n' % from_addr
    header += 'To: %s \n' % ','.join(to_addr_list)
    header += 'Cc: %s \n' % ','.join(cc_addr_list)
    header += 'Subject: %s \n \n' % subject
    message = header + message

    server = smtplib.SMTP(smtpserver)
    server.starttls()
    server.login(login,password)
    problems = server.sendmail(from_addr, to_addr_list, message)
    server.quit()

```

```

while True:
    try:
        if (GPIO.input(25) == True):
            sendemail(from_email, receipients_list,
                      cc_list, subject, message,
                      username, password, server)
            sleep(.01)
    except KeyboardInterrupt:
        exit()

```

7.6.3 Interfacing a Light Sensor (LDR) with Raspberry Pi

So far you have learned how to interface LED and switch with Raspberry Pi. Now let us look at an example of interfacing a Light Dependent Resistor (LDR) with Raspberry Pi and turning an LED on/off based on the light-level sensed.

Figure 7.11 shows the schematic diagram of connecting an LDR to Raspberry Pi. Connect one side of LDR to 3.3V and other side to a $1\mu\text{F}$ capacitor and also to a GPIO pin (pin 18 in this example). An LED is connected to pin 18 which is controlled based on the light-level sensed.

Box 7.5 shows the Python program for the LDR example. The *readLDR()* function returns a count which is proportional to the light level. In this function the LDR pin is set to output and low and then to input. At this point the capacitor starts charging through the resistor (and a counter is started) until the input pin reads high (this happens when capacitor voltage becomes greater than 1.4V). The counter is stopped when the input reads high. The final count is proportional to the light level as greater the amount of light, smaller is the LDR resistance and greater is the time taken to charge the capacitor.

■ Box 7.5: Python program for switching LED/Light based on reading LDR reading

```

import RPi.GPIO as GPIO
import time

GPIO.setmode(GPIO.BCM)
ldr_threshold = 1000
LDR_PIN = 18
LIGHT_PIN = 25

def readLDR(PIN):
    reading=0
    GPIO.setup(LIGHT_PIN, GPIO.OUT)
    GPIO.output(PIN, False)
    time.sleep(0.1)
    GPIO.setup(PIN, GPIO.IN)
    while (GPIO.input(PIN)==False):
        reading=reading+1
    return reading

def switchOnLight(PIN):
    GPIO.setup(PIN, GPIO.OUT)

```

```
GPIO.output(PIN, True)

def switchOffLight(PIN):
    GPIO.setup(PIN, GPIO.OUT)
    GPIO.output(PIN, False)

while True:
    ldr_reading = readLDR(LDR_PIN)
    if ldr_reading < ldr_threshold:
        switchOnLight(LIGHT_PIN)
    else:
        switchOffLight(LIGHT_PIN)

    time.sleep(1)
```

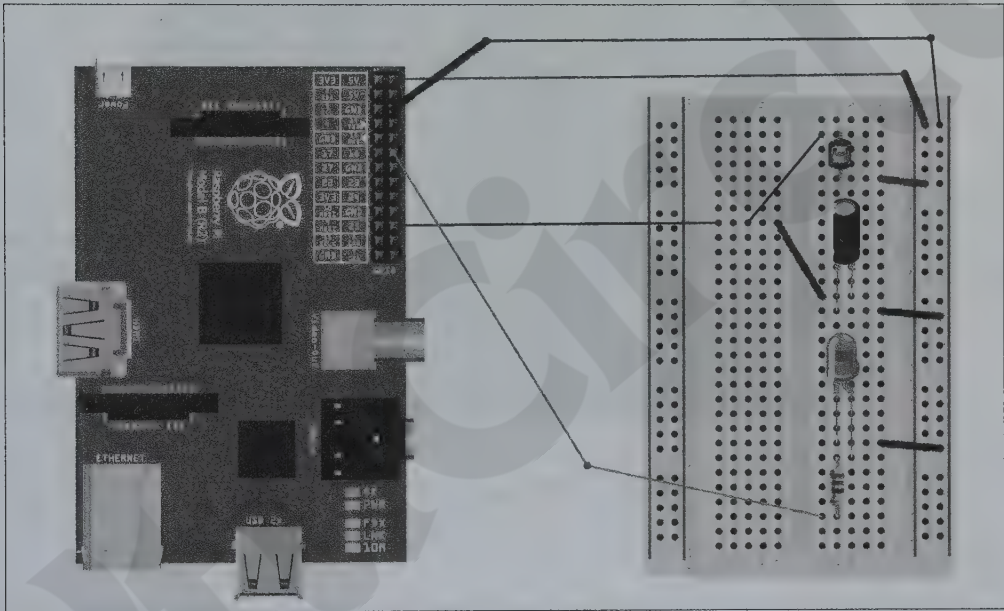


Figure 7.11: Interfacing LDR with Raspberry Pi

9.2 Home Automation

9.2.1 Smart Lighting

A design of a smart home automation system was described in Chapter-5 using the IoT design methodology. A concrete implementation of the system based on Django framework is described in this section. The purpose of the home automation system is to control the lights in a typical home remotely using a web application.

The system includes auto and manual modes. In auto mode, the system measures the light in a room and switches on the light when it gets dark. In manual mode, the system provides the option of manually and remotely switching on/off the light.

Figure 9.1 shows the deployment design of the home automation system. As explained in Chapter-5, the system has two REST services (mode and state) and a controller native service. Figures 9.2 and 9.3 show specifications of the mode and state REST services of the home automation system. The Mode service is a RESTful web service that sets mode to auto or manual (PUT request), or retrieves the current mode (GET request). The mode is updated to/retrieved from the database. The State service is a RESTful web service that sets the light appliance state to on/off (PUT request), or retrieves the current light state (GET request). The state is updated to/retrieved from the status database.

■ Box 9.1: Django model for mode and state REST services - models.py

```
from django.db import models

class Mode(models.Model):
    name = models.CharField(max_length=50)

class State(models.Model):
    name = models.CharField(max_length=50)
```

To start with the implementation of the system, we first map services to Django models. Box 9.1 shows the model fields for the REST services (state - on/off and mode - auto/manual).

■ Box 9.2: Serializers for mode and state REST services - serializers.py

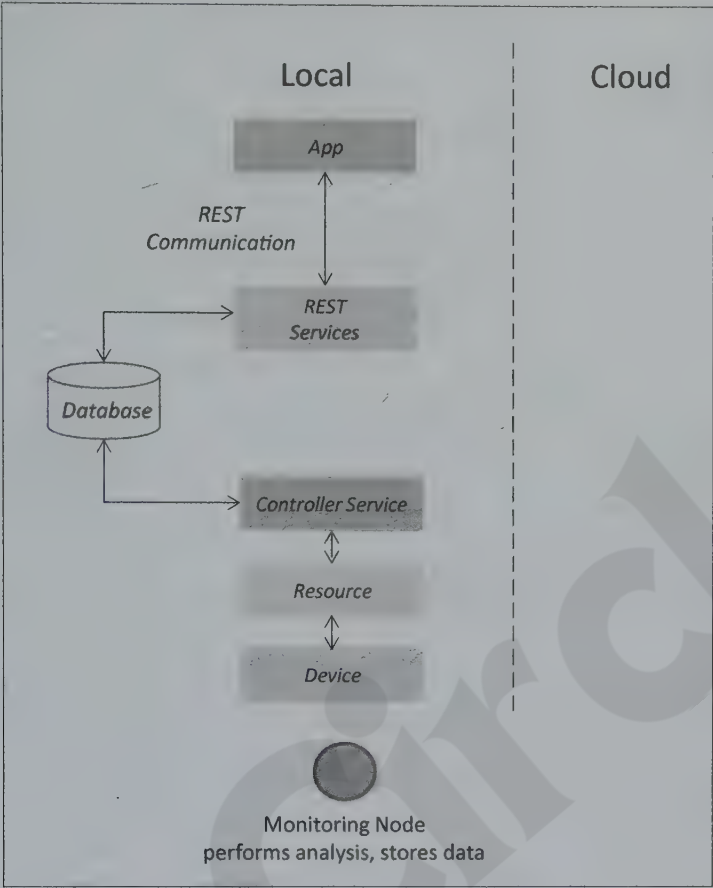


Figure 9.1: Deployment design of the home automation IoT system

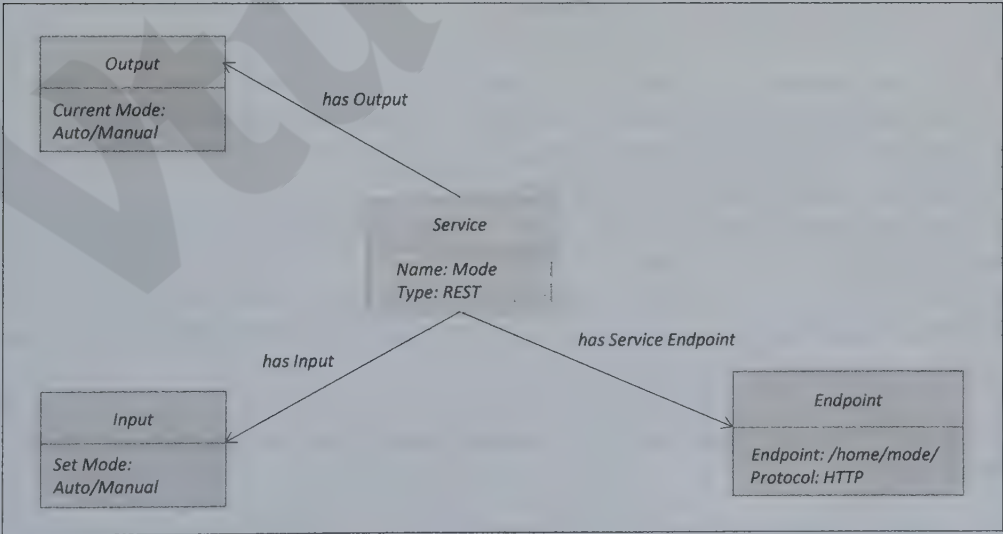


Figure 9.2: Service specification for home automation IoT system - mode service

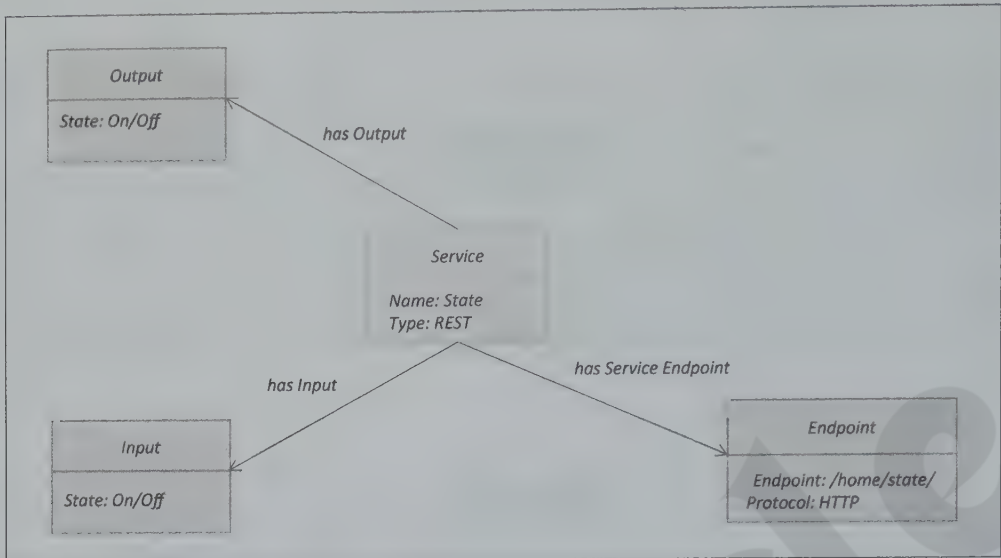


Figure 9.3: Service specification for home automation IoT system - state service

```

from myapp.models import Mode, State
from rest_framework import serializers

class ModeSerializer(serializers.HyperlinkedModelSerializer):
    class Meta:
        model = Mode
        fields = ('url', 'name')

class StateSerializer(serializers.HyperlinkedModelSerializer):
    class Meta:
        model = State
        fields = ('url', 'name')
  
```

After implementing the Django model, we implement the model serializers. Serializers allow complex data (such as model instances) to be converted to native Python datatypes that can then be easily rendered into JSON, XML or other content types. Box 9.2 shows the serializers for mode and state REST services.

After implementing the serializers, we write ViewSets for the Django models. ViewSets combine the logic for a set of related views in a single class. Box 9.3 shows the Django views for REST services and home automation application. The ViewSets for the models (ModeViewSet and StateViewSet) are included in the views file. The application view (home) is described later in this section.

■ Box 9.3: Django views for REST services and home automation application - views.py

```

from myapp.models import Mode, State
from rest_framework import viewsets
from django.shortcuts import render_to_response
from django.template import RequestContext
  
```

```
from myapp.serializers import ModeSerializer, StateSerializer
import requests
import json

class ModeViewSet(viewsets.ModelViewSet):
    queryset = Mode.objects.all()
    serializer_class = ModeSerializer

class StateViewSet(viewsets.ModelViewSet):
    queryset = State.objects.all()
    serializer_class = StateSerializer

def home(request):
    out=""
    currentmode='auto'
    currentstate='off'

    if 'on' in request.POST:
        values = "name": "on"
        r=requests.put('http://127.0.0.1:8000/state/1/',
            data=values, auth=('myuser', 'password'))
        result=r.text
        output = json.loads(result)
        out=output['name']
    if 'off' in request.POST:
        values = "name": "off"
        r=requests.put('http://127.0.0.1:8000/state/1/',
            data=values, auth=('myuser', 'password'))
        result=r.text
        output = json.loads(result)
        out=output['name']
    if 'auto' in request.POST:
        values = "name": "auto"
        r=requests.put('http://127.0.0.1:8000/mode/1/',
            data=values, auth=('myuser', 'password'))
        result=r.text
        output = json.loads(result)
        out=output['name']
    if 'manual' in request.POST:
        values = "name": "manual"
        r=requests.put('http://127.0.0.1:8000/mode/1/',
            data=values, auth=('myuser', 'password'))
        result=r.text
        output = json.loads(result)
        out=output['name']

    r=requests.get('http://127.0.0.1:8000/mode/1/',
        auth=('myuser', 'password'))
    result=r.text
    output = json.loads(result)
    currentmode=output['name']

    r=requests.get('http://127.0.0.1:8000/state/1/',
        auth=('myuser', 'password'))
    result=r.text
```

```

output = json.loads(result)
currentstate=output['name']

return render_to_response('lights.html', 'r':out,
'currentmode':currentmode, 'currentstate':currentstate,
context_instance=RequestContext(request))

```

Box 9.4 shows the URL patterns for the REST services and home automation application. Since ViewSets are used instead of views for the REST services, we can automatically generate the URL configuration by simply registering the viewsets with a router class. Routers automatically determine how the URLs for an application should be mapped to the logic that deals with handling incoming requests.

■ Box 9.4: Django URL patterns for REST services and home automation application - urls.py

```

from django.conf.urls import patterns, include, url
from django.contrib import admin
from rest_framework import routers
from myapp import views

admin.autodiscover()

router = routers.DefaultRouter()
router.register(r'mode', views.ModeViewSet)
router.register(r'state', views.StateViewSet)

urlpatterns = patterns("",
    url(r'^$', include(router.urls)),
    url(r'^api-auth/', include('rest_framework.urls',
        namespace='rest_framework')),
    url(r'^admin/', include(admin.site.urls)),
    url(r'^home/', 'myapp.views.home'),
)

```

Box 9.5 shows the code for the Django template for the home automation application.

■ Box 9.5: Django template for home automation application - index.html

```

<!DOCTYPE html>
<html>
<head>

</head>
<body>
<p>{{r}}<p>
<h3>State</h3>
<form action="" method="post">{% csrf_token %}
<input type="submit" name="on" value="on" />
<input type="submit" name="off" value="off" />
</form>
<br>

```

```

<h3>Mode</h3>
<form action="" method="post">{% csrf_token %}
<input type="submit" name="auto" value="auto" />
<input type="submit" name="manual" value="manual" />
</form>

</body>
</html>

```

Figure 9.4 shows a screenshot of the home automation web application.

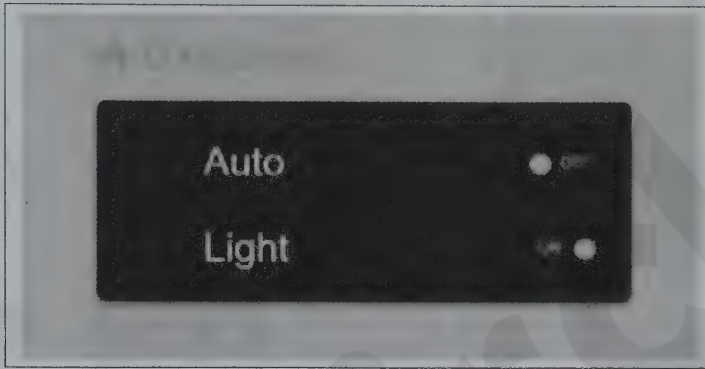


Figure 9.4: Home automation web application screenshot

Figure 9.5 shows a schematic diagram of the home automation IoT system. The devices and components used in this example are Raspberry Pi mini computer, LDR sensor and relay switch actuator.

Figure 9.6 shows the specification of the controller native service that runs on Raspberry Pi. When in auto mode, the controller service monitors the light level and switches the light on/off and updates the status in the status database. When in manual mode, the controller service, retrieves the current state from the database and switches the light on/off. A Python implementation of the controller service is shown in Box 9.6.

■ Box 9.6: Python code for controller native service - controller.py

```

import time
import datetime
import sqlite3
import spidev
import RPi.GPIO as GPIO

#Initialize SQLite
con = sqlite3.connect('database.sqlite')
cur = con.cursor()

#LDR channel on MCP3008
LIGHT_CHANNEL = 0

#GPIO Setup
GPIO.setmode(GPIO.BCM)

```

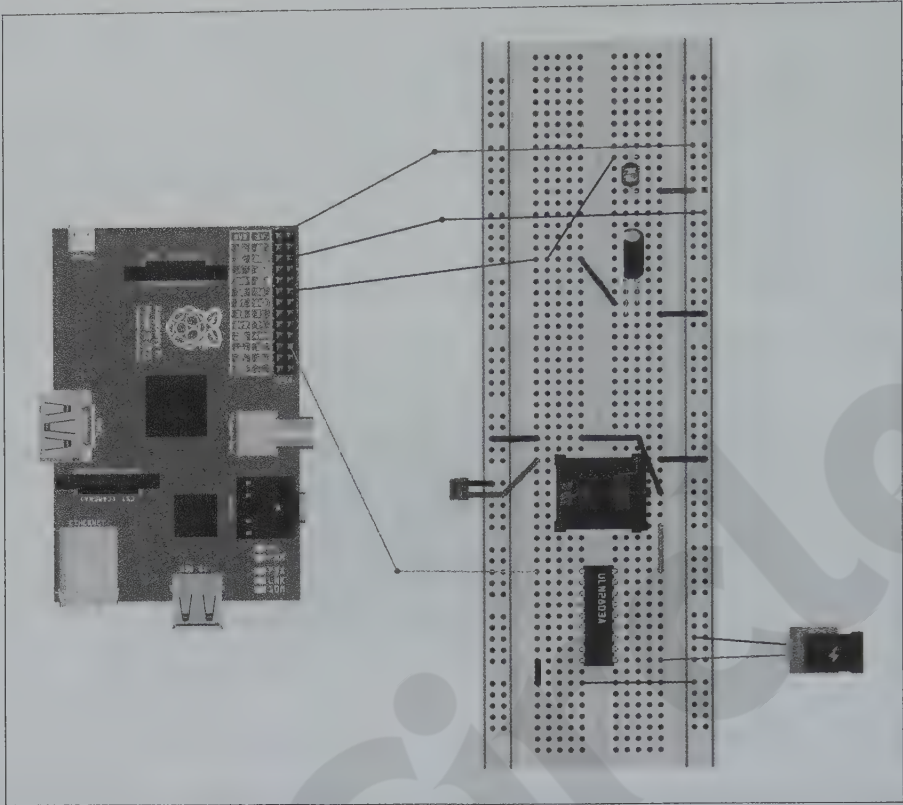


Figure 9.5: Schematic diagram of the home automation IoT system showing the device, sensor and actuator integrated

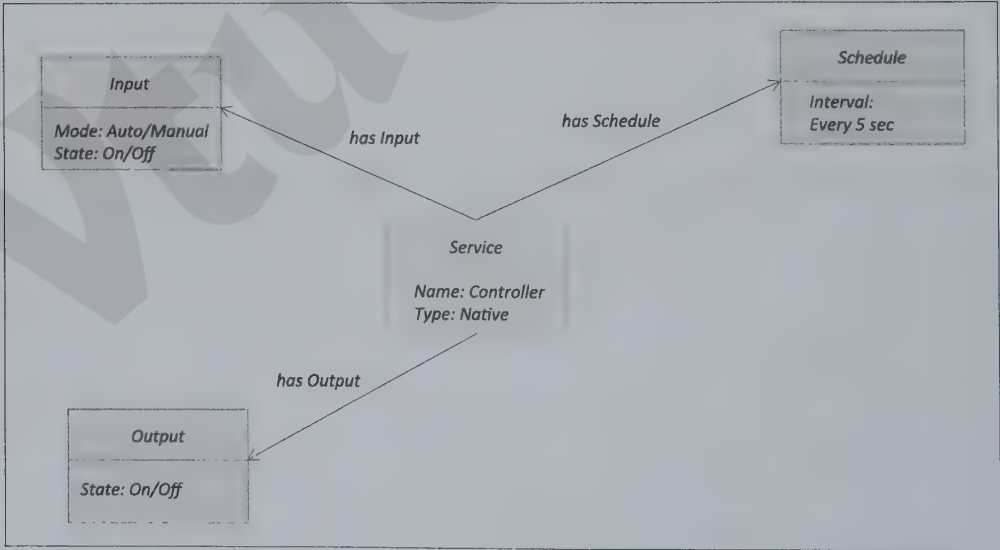


Figure 9.6: Controller service of the home automation IoT system

```
LIGHT_PIN = 25

# Open SPI bus
spi = spidev.SpiDev()
spi.open(0,0)

#Light Level Threshold
threshold=200

#Function to read LDR connected to MCP3008
def readLDR():
    light_level = ReadChannel(LIGHT_CHANNEL)
    lux = ConvertLux(light_level,2)
    return lux

#Function to convert LDR reading to Lux
def ConvertLux(data,places):
    R=10 #10k-ohm resistor connected to LDR
    volts = (data * 3.3) / 1023
    volts = round(volts,places)
    lux=500*(3.3-volts)/(R*volts)
    return lux

# Function to read SPI data from MCP3008 chip
def ReadChannel(channel):
    adc = spi.xfer2([1, (8+channel)<<4,0])
    data = ((adc[1]&3) << 8) + adc[2]
    return data

#Get current state from DB
def getCurrentMode():
    cur.execute('SELECT * FROM myapp_mode')
    data = cur.fetchone() # (1, u'auto')
    return data[1]

#Get current state from DB
def getCurrentState():
    cur.execute('SELECT * FROM myapp_state')
    data = cur.fetchone() # (1, u'on')
    return data[1]

#Store current state in DB
def setCurrentState(val):
    query='UPDATE myapp_state set name="'+val+'"'
    cur.execute(query)

def switchOnLight(PIN):
    GPIO.setup(PIN, GPIO.OUT)
    GPIO.output(PIN, True)

def switchOffLight(PIN):
    GPIO.setup(PIN, GPIO.OUT)
    GPIO.output(PIN, False)
```

```

def runManualMode():
    #Get current state from DB
    currentState=getCurrentState()
    if currentState=='on':
        switchOnLight(LIGHT_PIN)
    elif currentState=='off':
        switchOffLight(LIGHT_PIN)

def runAutoMode():
    #Read LDR
    lightlevel=readLDR()

    if lightlevel < ldr_threshold:
        switchOnLight(LIGHT_PIN)
    else:
        switchOffLight(LIGHT_PIN)

    print 'Manual'+ ' - '+getCurrentState()

#Controller main function
def runController():
    currentMode=getCurrentMode()
    if currentMode=='auto':
        runAutoMode()
    elif currentMode=='manual':
        runManualMode()

    return true

while True:
    runController()
    time.sleep(5)

```

9.2.2 Home Intrusion Detection

You got an overview of home intrusion detection systems in Chapter-2. A concrete implementation of a home intrusion detection system is described in this section. The purpose of the home intrusion detection system is to detect intrusions using sensors (such as PIR sensors and door sensors) and raise alerts, if necessary.

Figure 9.7 shows the process diagram for the home intrusion detection system. Each room in the home has a PIR motion sensor and each door has a door sensor. These sensors can detect motion or opening of doors. Each sensor is read at regular intervals and the motion detection or door opening events are stored and alerts are sent.

Figure 9.8 shows the domain model for the home intrusion detection system. The domain model includes physical entities for room and door and the corresponding virtual entities. The device in this example is a single-board mini computer which has PIR and door sensors attached to it. The domain model also includes the services involved in the system.

Figure 9.9 shows the information model for the home intrusion detection system. The information model defines the attributes of room and door virtual entities and their possible values. The room virtual entity has an attribute 'motion' and the door virtual entity has an attribute 'state'.

The next step is to define the service specifications for the system. The services are

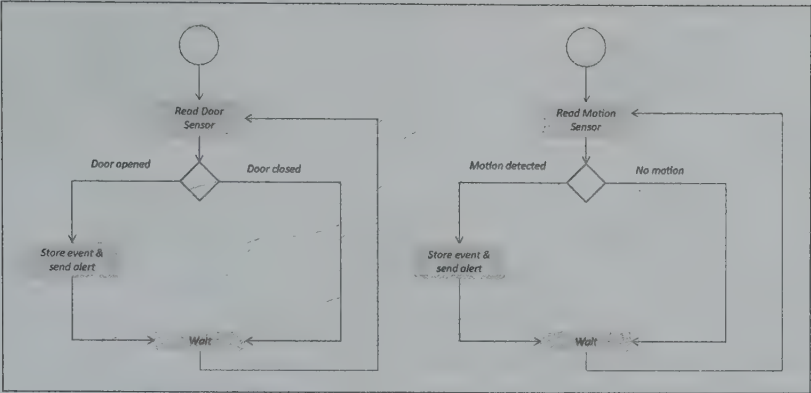


Figure 9.7: Process specification of the home intrusion detection IoT system

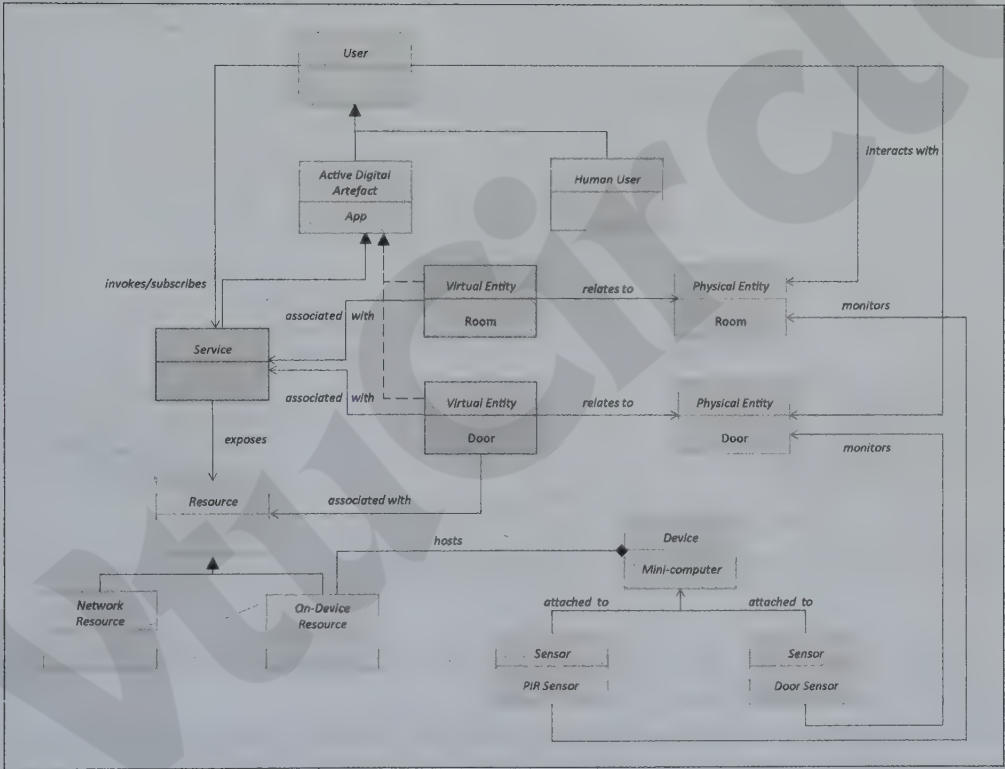


Figure 9.8: Domain model of the home intrusion detection IoT system

derived from the process specification and the information model. The system has three services - (1) a RESTful web service that retrieves the current state of a door from the database or sets the current state of a door to open/closed, (2) A RESTful web service that retrieves the current motion in a room or sets the motion of a room to yes/no, (3) a native controller service that runs on the device and reads the PIR and door sensors and calls the REST services for updating the state of rooms and doors in the database. Figures 9.10, Figures 9.11 and 9.12 show specifications of the web services and the controller service.

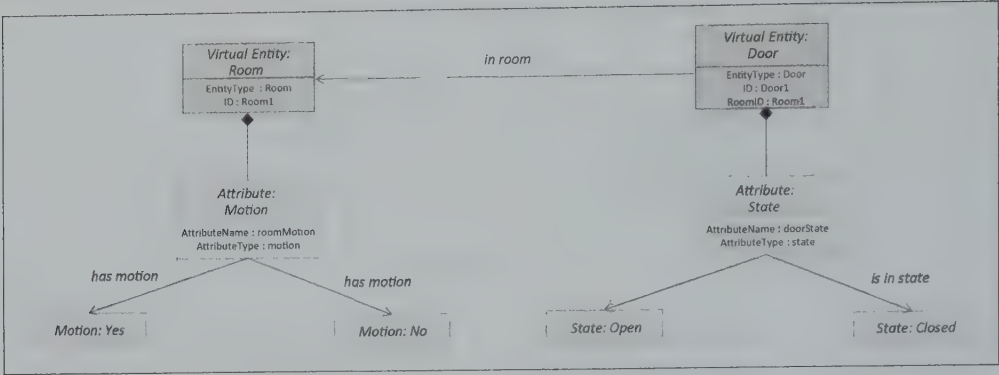


Figure 9.9: Information model of the home intrusion detection IoT system

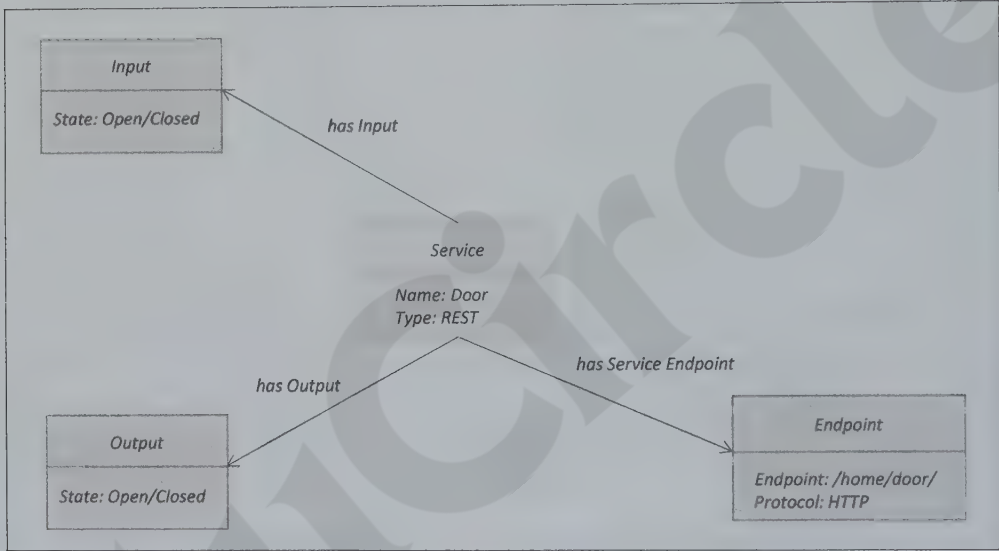


Figure 9.10: Service specification for the home intrusion detection IoT system - door service

Figure 9.13 shows the deployment design for the home intrusion detection system. Recall that this is a level-2 IoT system.

The functional view and the operational view specifications for home intrusion detection system are shown in Figure 9.14. Various options pertaining to the system deployment and operation and their mapping to the corresponding functional groups is shown in Figure 9.14. The system uses Django framework for web application and REST service. The Django web application is backed by a MySQL database. The IoT device used for this example is Raspberry Pi along with the PIR and door sensors. Figure 9.15 shows a schematic diagram of the home intrusion detection system and Figure 9.16 shows how the sensors are deployed in a parking.

Let us now look at the implementation of the web application and services for the system. Box 9.7 shows the model fields for the room and door REST service. After implementing the Django model, we implement the model serializers that allows model instances to be converted to native Python datatypes. Box 9.8 shows the serializers for room and door REST

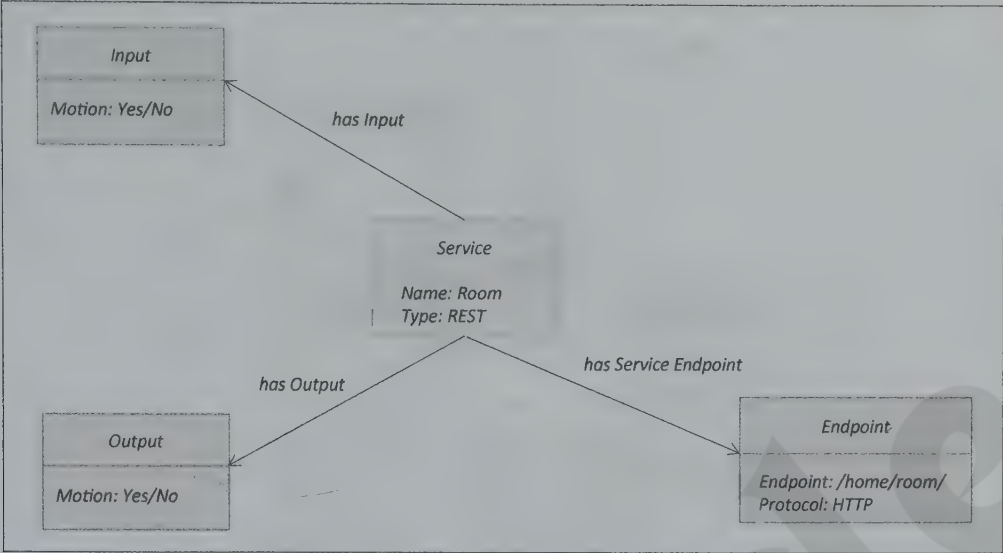


Figure 9.11: Service specification for the home intrusion detection IoT system - room service

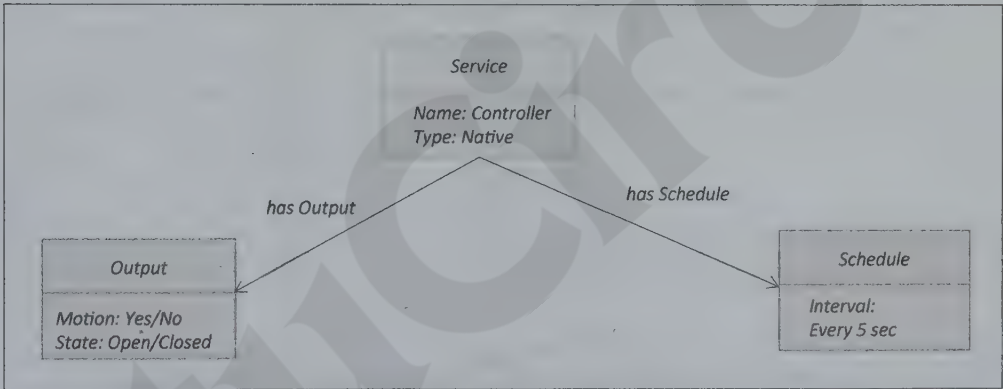


Figure 9.12: Controller service of the home intrusion detection IoT system

services.

■ **Box 9.7: Django model for room and door REST services - models.py**

```
from django.db import models

class Room(models.Model):
    name = models.CharField(max_length=50)
    state = models.CharField(max_length=50)
    timestamp = models.CharField(max_length=50)
    pin = models.CharField(max_length=5)

class Door(models.Model):
    name = models.CharField(max_length=50)
    state = models.CharField(max_length=50)
```

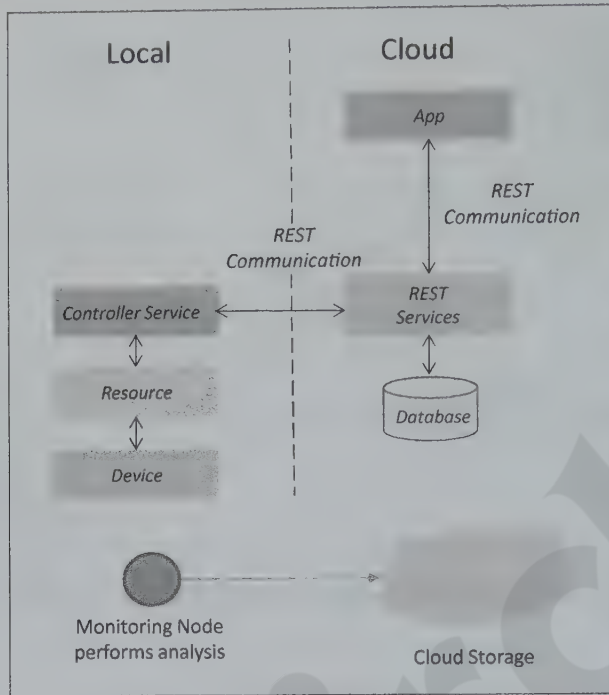


Figure 9.13: Deployment design for the home intrusion detection IoT system

```
timestamp = models.CharField(max_length=50)
pin = models.CharField(max_length=5)
```

■ Box 9.8: Serializers for room and door REST services - serializers.py

```
from myapp.models import Room, Door
from rest_framework import serializers

class RoomSerializer(serializers.HyperlinkedModelSerializer):
    class Meta:
        model = Room
        fields = ('url', 'name', 'state', 'timestamp', 'pin')

class DoorSerializer(serializers.HyperlinkedModelSerializer):
    class Meta:
        model = Door
        fields = ('url', 'name', 'state', 'timestamp', 'pin')
```

Box 9.9 shows the Django views for REST services and home intrusion detection application. The ViewSets for the models (RoomViewSet and RoomViewSet) are included in the views file. The *home* view renders the content for the home intrusion detection application home page that displays the status of each room. Notice that a request is sent to the state REST service to obtain the state of a room.

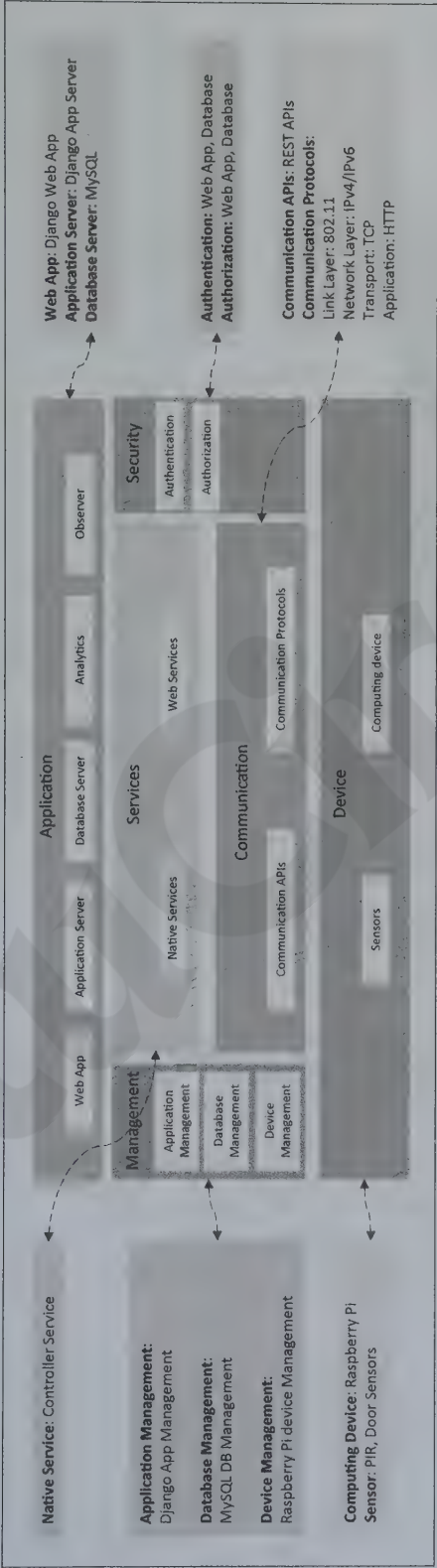


Figure 9.14: Functional & operational view specifications for home intrusion detection system

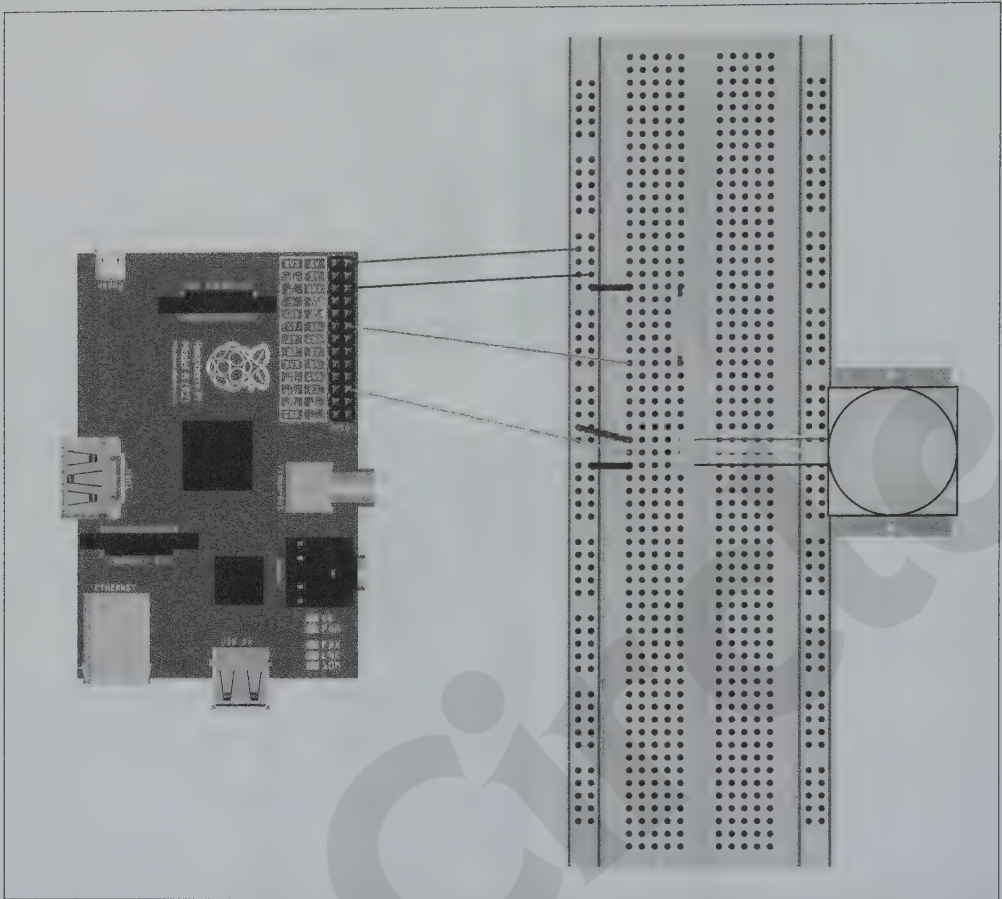


Figure 9.15: Schematic diagram of the home intrusion detection IoT system prototype, showing the device and ultrasonic sensor

■ Box 9.9: Django views for REST services and home intrusion detection application - views.py

```
from myapp.models import Room, Door
from rest_framework import viewsets
from django.shortcuts import render_to_response
from django.template import RequestContext
from myapp.serializers import RoomSerializer, DoorSerializer
import requests
import json

class RoomViewSet(viewsets.ModelViewSet):
    queryset = Room.objects.all()
    serializer_class = RoomSerializer

class DoorViewSet(viewsets.ModelViewSet):
    queryset = Door.objects.all()
    serializer_class = DoorSerializer
```

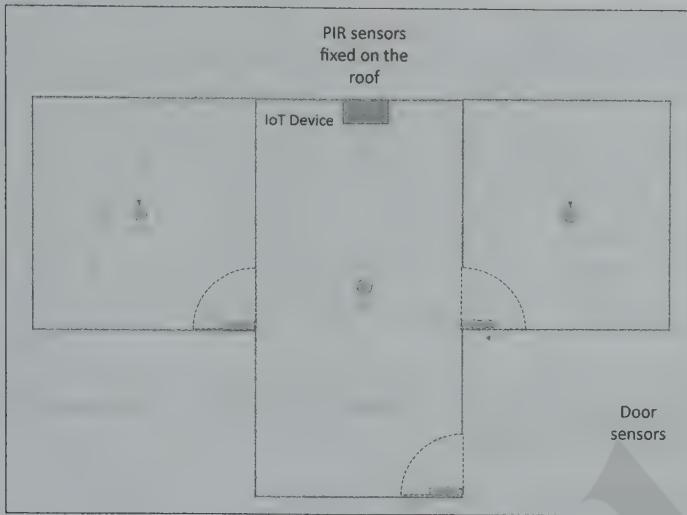


Figure 9.16: Deployment of sensors for home intrusion detection system

```
def home(request):
    r=requests.get('http://127.0.0.1:8000/room/', auth=('username',
'password'))
    result=r.text
    output = json.loads(result)
    roomCount = output['count']

    r=requests.get('http://127.0.0.1:8000/door/', auth=('username',
'password'))
    result=r.text
    output = json.loads(result)
    doorCount = output['count']

    roomsDict={}
    for i in range (0, roomCount):
        r=requests.get('http://127.0.0.1:8000/room/'+str(i+1)+'/',
            auth=('username', 'password'))
        result=r.text
        output = json.loads(result)
        roomName=output['name']
        roomState=output['state']
        roomTimestamp=output['timestamp']
        roomsDict[roomName] = [roomState, roomTimestamp]

    doorsDict={}
    for i in range (0, doorCount):
        r=requests.get('http://127.0.0.1:8000/door/'+str(i+1)+'/',
            auth=('username', 'password'))
        result=r.text
        output = json.loads(result)
        doorName=output['name']
        doorState=output['state']
```

```

doorTimestamp=output['timestamp']
doorsDict[doorName] = [doorState, doorTimestamp]

return render_to_response('index.html',
{'roomsDict':roomsDict, 'doorsDict':doorsDict},
context_instance=RequestContext(request))

```

Box 9.10 shows the URL patterns for the REST services and home intrusion detection application. Since ViewSets are used instead of views for the REST services, we can automatically generate the URL configuration by simply registering the viewsets with a router class.

■ Box 9.10: Django URL patterns for REST services and home intrusion detection application - urls.py

```

from django.conf.urls import patterns, include, url
from django.contrib import admin
from rest_framework import routers
from myapp import views

admin.autodiscover()

router = routers.DefaultRouter()
router.register(r'room', views.RoomViewSet)
router.register(r'door', views.DoorViewSet)

urlpatterns = patterns("",
    url(r'^$', include(router.urls)),
    url(r'^api-auth/', include('rest_framework.urls',
        namespace='rest_framework')),
    url(r'^admin/', include(admin.site.urls)),
    url(r'^home/', 'myapp.views.home'),
)

```

Box 9.11 shows the code for the Django template for the home intrusion detection application. This template is rendered by the *home* view. Figure 9.17 shows a screenshot of the home intrusion detection web application.

■ Box 9.11: Django template for home intrusion detection application - index.html

```

<html>
<head>
<meta charset="utf-8">
<meta http-equiv="X-UA-Compatible" content="IE=edge,chrome=1">
<title>Home Intrusion Detection App</title>
<link rel="stylesheet" href="/static/css/style.css">
</head>
<body>

<div class="app-container">

```

```

<header class="app-header clearfix">
<h1 class="app-logo js-app-title icon-home">Dashboard</h1>
<div class="app-state"><span class="app-loading-loader">
</span></div>
<center>
<h4>Home #: 123</h4>
</center>
</header>

<div role="main" class="app-content clearfix">
<div class="app-loading"><span class="app-loading-loader">
</span></div>
<div class="app-content-inner">

<form class="dashboard-control js-form clearfix">
<fieldset>
<div class="field clearfix">
<center><h4>Rooms</h4></center>
<table width = "90%" border="1">
{% for key,val in roomsDict.items %}
<tr>
<td width="50%">{{key}}
<br>
<center>
{% if val.0 == 'no' %}

{% else %}

{% endif %}
<br>
<p>Last Updated:  {{val.1}}</p>
</center>
</td>
</tr>
{% endfor %}
</table>
</div></div>
</fieldset>
</form></div>
<br>

<div role="main" class="app-content clearfix">
<div class="app-loading"><span class="app-loading-loader">
</span></div>
<div class="app-content-inner">

<form class="dashboard-control js-form clearfix">
<fieldset>
<div class="field clearfix">
<center><h4>Doors</h4></center>
<table width = "90%" border="1">
{% for key,val in doorsDict.items %}
<tr>
<td width="50%">{{key}}

```

```

<br>
<center>
{% if val.0 == 'closed' %}

{% else %}

{% endif %}
<br>
<p>Last Updated: {{val.1}}</p>
</center>
</td>
</tr>
{% endfor %}
</table>
</div>
</div>
</fieldset>
</form>
</div>
</div>
<script src="https://ajax.googleapis.com/ajax/libs/
jquery/1.8.2/jquery.min.js"></script>
<script src="/static/js/script.js"></script>

</body> </html>

```

A Python implementation of the controller service native service that runs on Raspberry Pi, is shown in Box 9.12. The *runController* function is called every second and the readings of the PIR and door sensors are obtained. The current states of the room and door are then updated by sending a PUT request to the corresponding REST services.

■ **Box 9.12: Python code for controller native service - intrusion detection - controller.py**

```

import RPi.GPIO as GPIO
import time
import sys

GPIO.setmode(GPIO.BCM)

global PIR_SENSOR_PIN

global DOOR_SENSOR_PIN
def readingPIRSensor():

    if GPIO.input(PIR_SENSOR_PIN):
        return 1
    else:
        return 0

def readingDoorSensor():
    if GPIO.input(DOOR_SENSOR_PIN):

```

```

        return 1
    else:
        return 0

def runController():
    pirState = readingPIRSensor()
    if pinState == 1:
        setPIRState('yes')
    else:
        setPIRState('no')

    doorState = readingDoorSensor()
    if doorState == 1:
        setDoorState('open')
    else:
        setDoorState('closed')

def setPIRState(val):
    values = {"state": val, "timestamp": str(time.time())}
    r=requests.put('http://127.0.0.1:8000/room/1/', data=values,
        auth=('username', 'password'))
def setDoorState(val):
    values = {"state": val, "timestamp": str(time.time())}
    r=requests.put('http://127.0.0.1:8000/door/1/', data=values,
        auth=('username', 'password'))

def setupController():
    r=requests.get('http://127.0.0.1:8000/config/1/', data=values,
        auth=('username', 'password'))
    configStr=r.text
    config = json.loads(configStr)
    global PIR_SENSOR_PIN = config['room1']
    global DOOR_SENSOR_PIN = config['door1']
    GPIO.setup(PIR_SENSOR_PIN,GPIO.IN)
    GPIO.setup(DOOR_SENSOR_PIN,GPIO.IN, pull_up_down=GPIO.PUD_UP)

setupController()
while True:
    runController()
    time.sleep(1)

```

9.3 Cities

9.3.1 Smart Parking

You got an overview of smart parking systems in Chapter-2. A concrete implementation of a smart parking IoT system is described in this section.

The purpose of a smart parking system is to detect the number of empty parking slots and send the information over the Internet to smart parking application backends. These applications can be accessed by drivers from smartphones, tablets or from in-car navigation systems. In smart parking, sensors are used for each parking slot, to detect whether the slot is empty or occupied. This information is aggregated by a local controller and then sent over

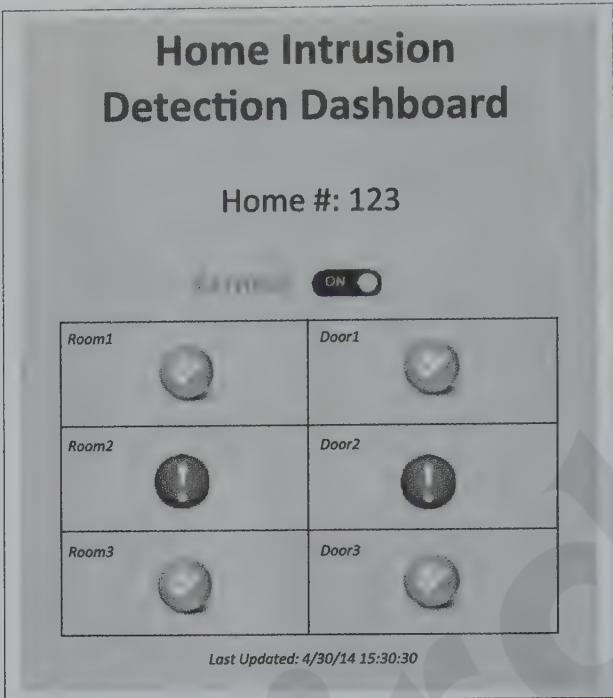


Figure 9.17: Home intrusion detection web application screenshot

the Internet to a server.

Figure 9.18 shows the process diagram for the smart parking system. Each parking slot has an ultrasonic sensor fixed above, which can detect the presence of a vehicle in the slot. Each sensor is read at regular intervals and the state of the parking slot (empty or occupied) is updated in a database.

Figure 9.19 shows the domain model for the smart parking system. The domain model includes a physical entity for the parking slot and the corresponding virtual entity. The device in this example is a single-board mini computer which has ultrasonic sensor attached to it. The domain model also includes the services involved in the system.

Figure 9.20 shows the information model for the smart parking system. The information model defines the attribute (state) of the parking slot virtual entity with two possible values (empty or occupied).

The next step is to define the service specifications for the system. The services are derived from the process specification and the information model. The smart parking system has two services - (1) a service that monitors the parking slots (using ultrasonic sensors) and updates the status in a database on the cloud (REST web service), (2) a service that retrieves the current state of the parking slots (controller native service). Figures 9.21 and 9.22 show specifications of the controller and state services of the smart parking system.

The functional view and the operational view specifications for smart parking system are similar to the specifications for the home intrusion detection system shown in Figure 9.14. The system uses Django framework for web application and REST service - both of which you learned about from earlier chapters of this book. The Django web application is supported

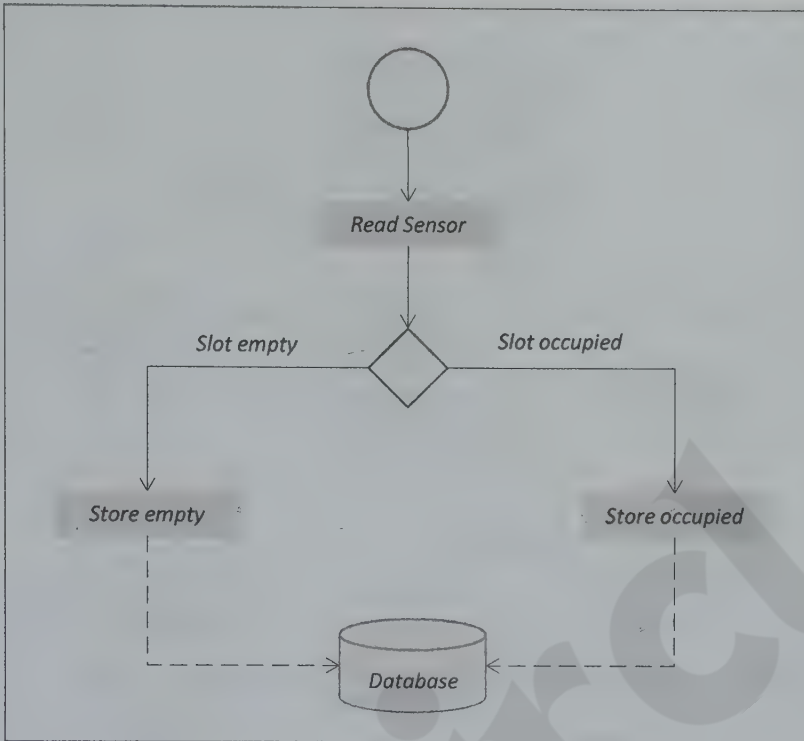


Figure 9.18: Process specification for the smart parking IoT system

by a MySQL database. The IoT device used for this example is Raspberry Pi along with the ultrasonic sensors. Figure 9.23 shows how the sensors are deployed in a parking and Figure 9.24 shows a schematic diagram of the smart parking system.

Let us now look at the implementation of the web application and services for the smart parking system. Box 9.13 shows the model fields for the state REST service. After implementing the Django model, we implement the model serializer which allows model instances to be converted to native Python datatypes. Box 9.14 shows the serializer for state REST service.

■ Box 9.13: Django model for REST service - models.py

```
from django.db import models

class State(models.Model):
    name = models.CharField(max_length=50)
```

■ Box 9.14: Serializer for REST service - serializers.py

```
from myapp.models import State
from rest_framework import serializers
```

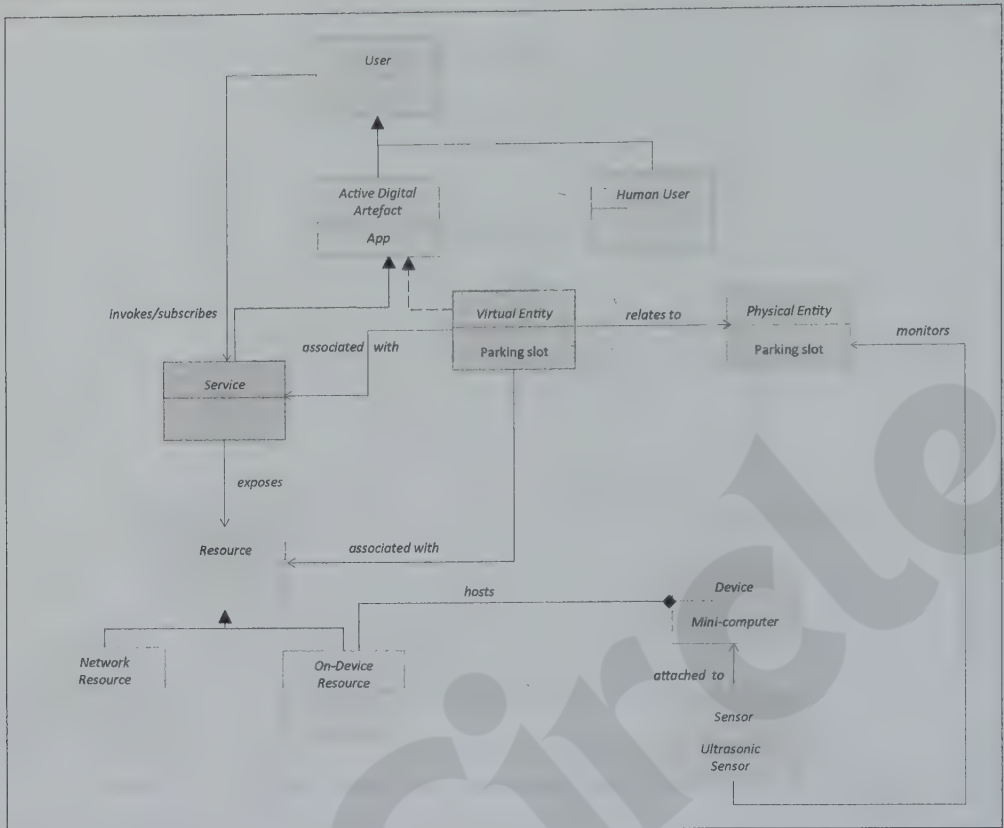


Figure 9.19: Domain model of the smart parking IoT system

```
class StateSerializer(serializers.HyperlinkedModelSerializer):
    class Meta:
        model = State
        fields = ('url', 'name')
```

Box 9.15 shows the Django views for REST services and smart parking application. The ViewSets for the model (StateViewSet) are included in the views file. The *home* view renders the content for the smart parking application home page that displays the status of the parking slots. Notice that a request is sent to the state REST service to obtain the state of a parking slot. The code shown in this example is for a trivial case of a one-slot parking.

■ Box 9.15: Django views for REST service and smart parking application - views.py

```
from myapp.models import State
from rest_framework import viewsets
from django.shortcuts import render_to_response
from django.template import RequestContext
from myapp.serializers import StateSerializer
import requests
import json
```

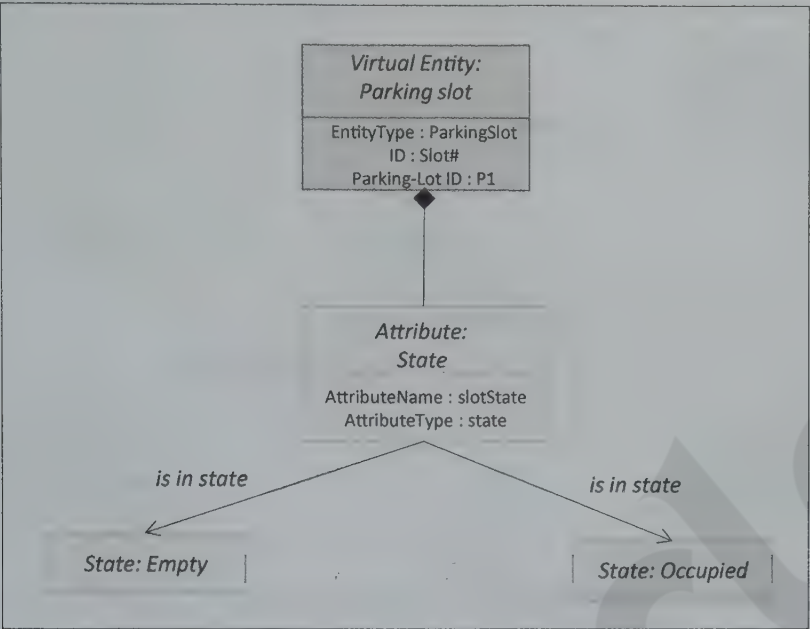


Figure 9.20: Information model of the smart parking IoT system

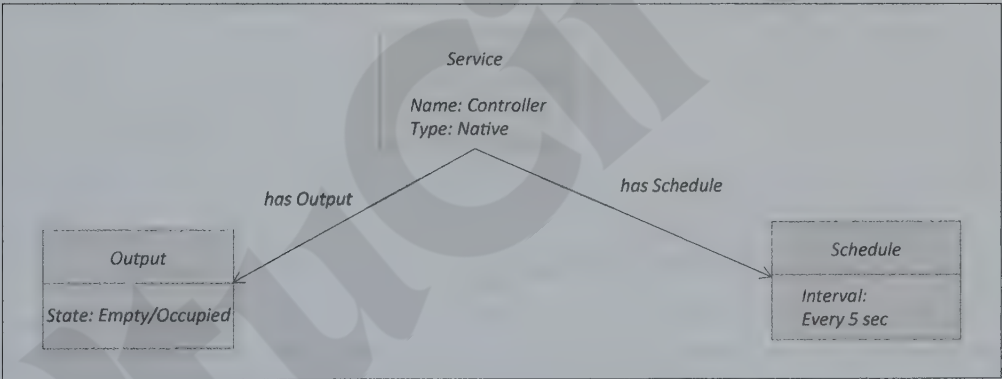


Figure 9.21: Controller service of the smart parking IoT system

Internet of Things - A Hands-On Approach

```
class StateViewSet(viewsets.ModelViewSet):
    queryset = State.objects.all()
    serializer_class = StateSerializer

def home(request):
    currentstate='off'
    r=requests.get('http://127.0.0.1:8000/state/1/',
    auth=('username', 'password'))
    result=r.text
    output = json.loads(result)
    currentstate=output['name']
```

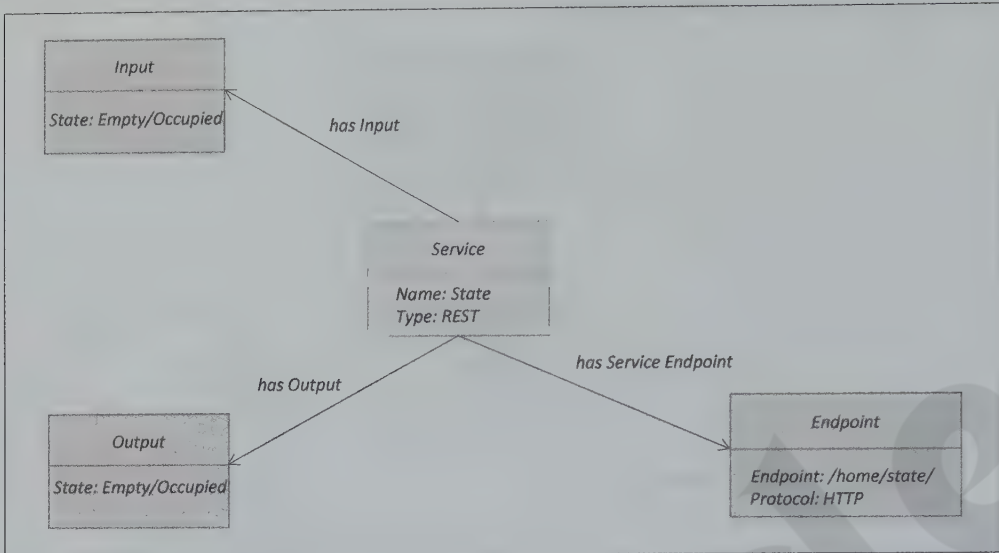


Figure 9.22: Service specification for the smart parking IoT system - state service

```

if currentstate=='empty':
    occupiedCount=0
    emptyCount=1
else:
    occupiedCount=1
    emptyCount=0

return render_to_response('index.html',
{'currentstate':currentstate, 'occupiedCount':
occupiedCount, 'emptyCount': emptyCount},
context_instance=RequestContext(request))
  
```

Box 9.16 shows the URL patterns for the REST service and smart parking application. Since ViewSets are used instead of views for the REST service, we can automatically generate the URL configuration by simply registering the viewsets with a router class.

■ Box 9.16: Django URL patterns for REST service and smart parking application - urls.py

```

from django.conf.urls import patterns, include, url
from django.contrib import admin
from rest_framework import routers
from myapp import views

admin.autodiscover()

router = routers.DefaultRouter()
router.register(r'state', views.StateViewSet)

urlpatterns = patterns("",
    url(r'^', include(router.urls)),
  
```

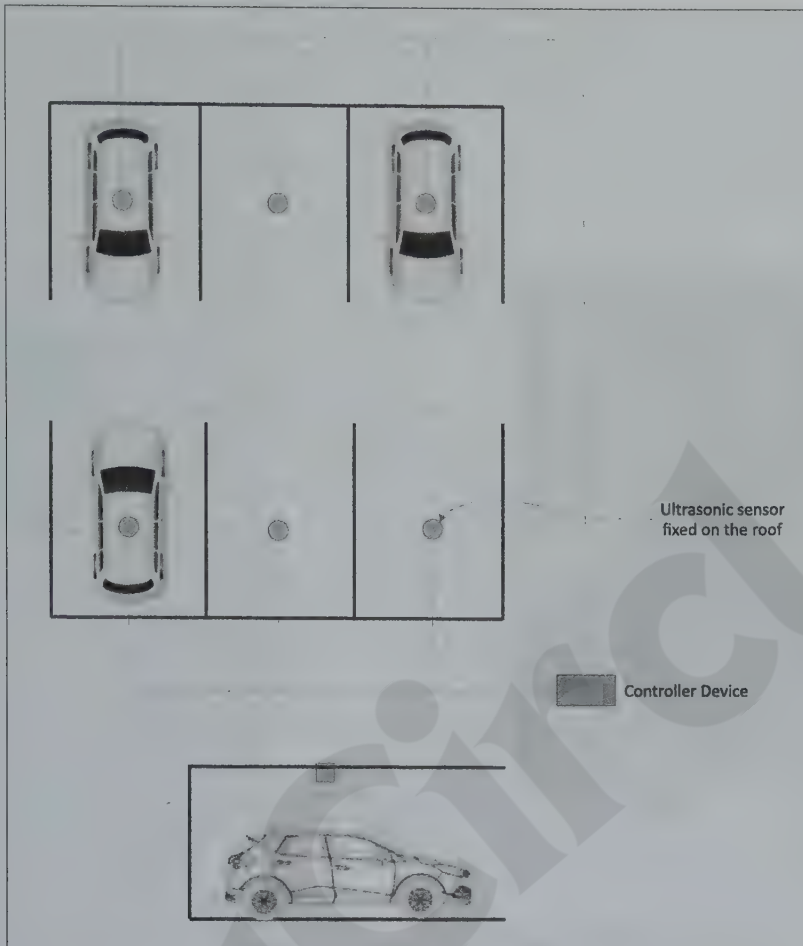


Figure 9.23: Deployment of sensors for smart parking system

```
url(r'^api-auth/', include('rest_framework.urls',
namespace='rest_framework')),
url(r'^admin/', include(admin.site.urls)),
url(r'^home/', 'myapp.views.home'),
)
```

Box 9.17 shows the code for the Django template for the smart parking application. This template is rendered by the *home* view.

■ Box 9.17: Django template for smart parking application - index.html

```
<html>
<head>
<meta charset="utf-8">
<meta http-equiv="X-UA-Compatible" content="IE=edge,chrome=1">
<title>Smart Parking App</title>
<link rel="stylesheet" href="/static/css/style.css">
```

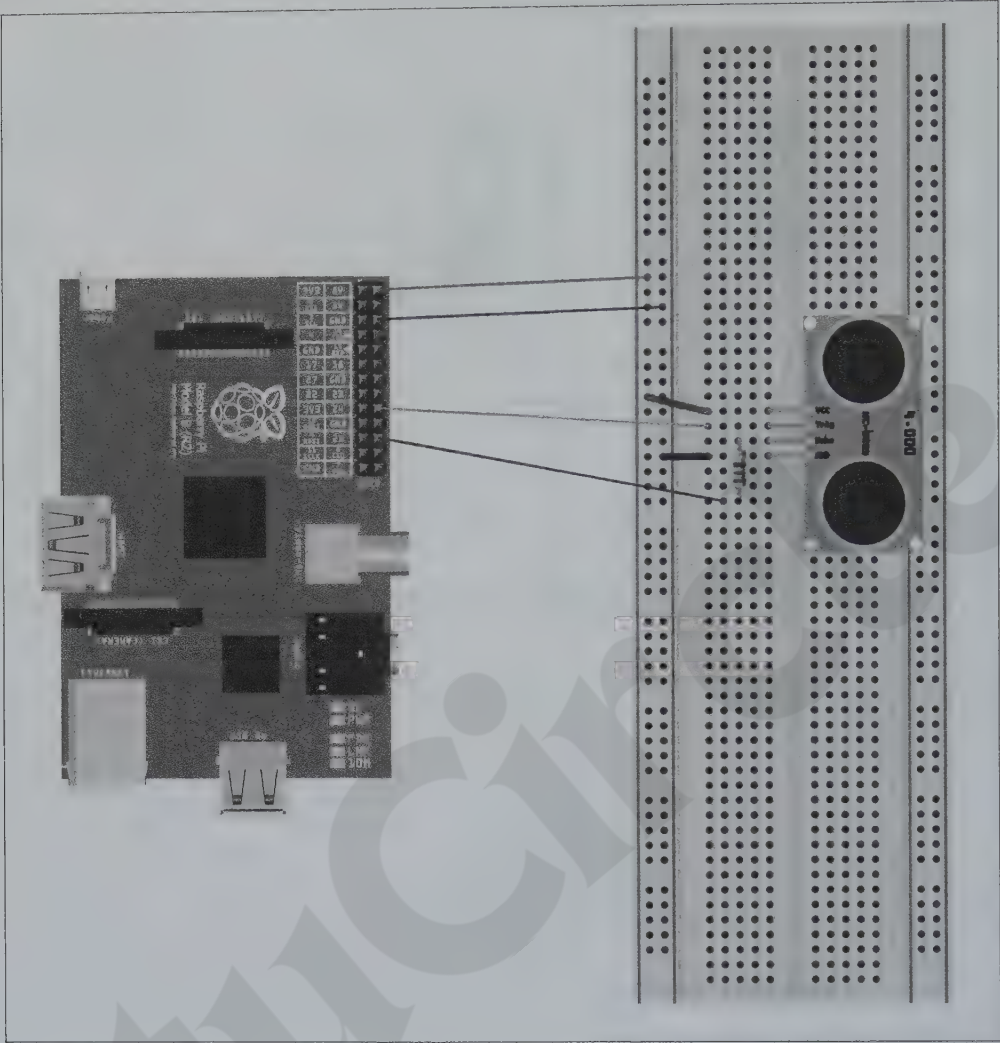


Figure 9.24: Schematic diagram of the smart parking IoT system prototype, showing the device and ultrasonic sensor

```
</head>
<body>
<div class="app-container">
<header class="app-header clearfix">
<h1 class="app-logo js-app-title icon-home">Smart Parking
Dashboard</h1>
<div class="app-state"><span
class="app-loading-loader"></span></div>
<center>
<h4>Parking Lot #: 123</h4>
<h4>Empty: {{emptyCount}} Occupied: {{occupiedCount}}</h4>
</center>
</header>
<div role="main" class="app-content clearfix">
```

```

<div class="app-loading"><span class="app-loading-loader"></span></div>
<div class="app-content-inner">
<form class="dashboard-control js-form clearfix">
<fieldset>

<div class="field clearfix">
<table width = "90%" border="1">
<tr>
<td width="100%">1
<br>
<center>
{% if currentstate == 'empty' %}

{% else %}

{% endif %}
</center>
</td>
</tr>
</table>
</div>
</div>
</fieldset>
</form>
</div></div></div>

<script src="https://ajax.googleapis.com/ajax/libs/
jquery/1.8.2/jquery.min.js"></script>
<script src="/static/js/script.js"></script>
</body></html>

```

Figure 9.25 shows a screenshot of the smart parking web application.

A Python implementation of the controller service native service that runs on Raspberry Pi, is shown in Box 9.18. The *runController* function is called every second and the reading of the ultrasonic sensor is obtained. If the distance returned by the sensor is less than a threshold, the slot is considered to be occupied. The current state of the slot is then updated by sending a PUT request to the state REST service.

■ Box 9.18: Python code for controller native service - smart parking - controller.py

```

import RPi.GPIO as GPIO
import time
import sys

GPIO.setmode(GPIO.BCM)
SENSOR_PIN = 27
TRIGGER_PIN=17
threshold = 10000

def readUltrasonicSensor():
    GPIO.setup(TRIGGER_PIN, GPIO.OUT)
    GPIO.setup(SENSOR_PIN, GPIO.IN)

```



Figure 9.25: Samrt Parking web application screenshot

```
GPIO.output(TRIGGER_PIN, GPIO.LOW)
time.sleep(0.3)
GPIO.output(TRIGGER_PIN, True)
time.sleep(0.00001)
GPIO.output(TRIGGER_PIN, False)
while GPIO.input(SENSOR_PIN) == 0:
    signaloff = time.time()

while GPIO.input(SENSOR_PIN) == 1:
    signalon = time.time()

timepassed = signalon - signaloff
distance = timepassed * 17000
if distance < threshold:
    return 1
else:
    return 0

def runController():
    pinState = readUltrasonicSensor()
    if pinState == 1:
        setCurrentState('occupied')
    else:
        setCurrentState('empty')

def setCurrentState(val):
    values = {"name": val}
    r=requests.put('http://127.0.0.1:8000/state/1/',
```

```
    data=values, auth=('username', 'password'))  
  
while True:  
    runController()  
    time.sleep(1)
```

9.5 Agriculture

9.5.1 Smart Irrigation

Smart irrigation systems use IoT devices and soil moisture sensors to determine the amount of moisture in the soil and release the flow of water through the irrigation pipes only when the moisture levels go below a predefined threshold. Data on the moisture levels is also collected in the cloud where it is analyzed to plan watering schedules.

An implementation of a smart irrigation system is described in this section. The deployment design for the system is similar to the deployment shown in Figure 9.26. The system consists of multiple nodes placed in different locations for monitoring soil moisture in a field. The end nodes send the data to the cloud and the data is stored in a cloud database. A cloud-based application is used for visualizing the data. Figure 9.36 shows a schematic diagram of smart irrigation system end-node. The end node includes a Raspberry Pi mini-computer and soil moisture sensor. A solenoid valve is used to control the flow of water through the irrigation pipe. When the moisture level goes below a threshold, the valve is opened to release water. Box 9.30 shows the Python code for the controller native service for the smart irrigation system.

■ **Box 9.30: Python code for controller native service - smart irrigation system**

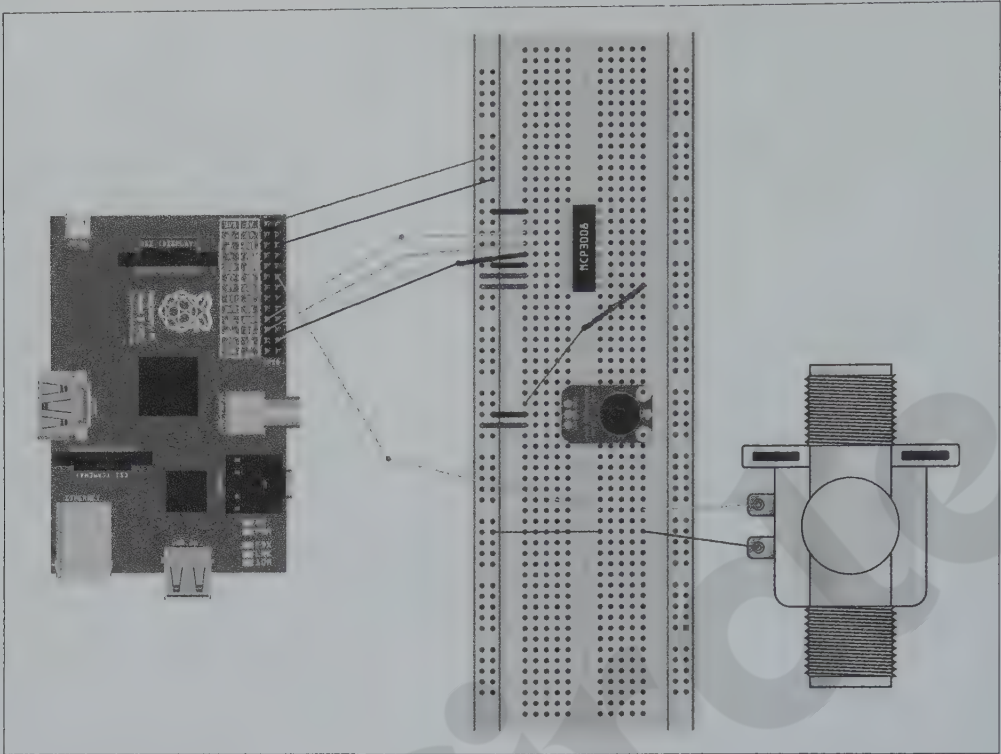


Figure 9.36: Schematic diagram of a smart irrigation system end-node showing the device and sensor

```
import time
import datetime
import sys
import json
import requests
import xively
import subprocess
import spidev

global mositure_datastream
#LDR channel on MCP3008
mositure_channel = 0

GPIO.setmode(GPIO.BCM)
TRIGGER_PIN=18
threshold = 10

# Open SPI bus
spi = spidev.SpiDev()
spi.open(0,0)

#Initialize Xively Feed
FEED_ID = "enter feed id"
API_KEY = "enter key"
```

```

api = xively.XivelyAPIClient(API_KEY)

# Function to read SPI data from MCP3008 chip
def ReadChannel(channel):
    adc = spi.xfer2([1, (8+channel)«4,0])
    data = ((adc[1]&3) « 8) + adc[2]
    return data

#Function to read sensor connected to MCP3008
def readMositure():
    level = ReadChannel(mositure_channel)
    return level

#Controller main function
def runController():
    global mositure_datastream

    level=readMositure()

    #Check moisture level
    if (level<threshold):
        GPIO.output(TRIGGER_PIN, True)
    else:
        GPIO.output(TRIGGER_PIN, False)

    mositure_datastream.current_value = level
    mositure_datastream.at = datetime.datetime.utcnow()

    print "Updating Xively feed with mositure:  %s" % mositure
    try:
        mositure_datastream.update()
    except requests.HTTPError as e:
        print "HTTPError({0}):  {1}".format(e.errno, e.strerror)

#Function to get existing or create new Xively data stream
def get_mosituredatastream(feed):
    try:
        datastream = feed.datastreams.get("mositure")
        return datastream
    except:
        datastream = feed.datastreams.create("mositure", tags="mositure")
        return datastream

#Controller setup function
def setupController():
    global mositure_datastream

    feed = api.feeds.get(FEED_ID)

    feed.location.lat="30.733315"
    feed.location.lon="76.779418"

```

```
feed.tags="Soil Moisture"
feed.update()

mositure_datastream = get_mosituredatastream(feed)
mositure_datastream.max_value = None
mositure_datastream.min_value = None

setupController()
while True:
    runController()
    time.sleep(10)
```